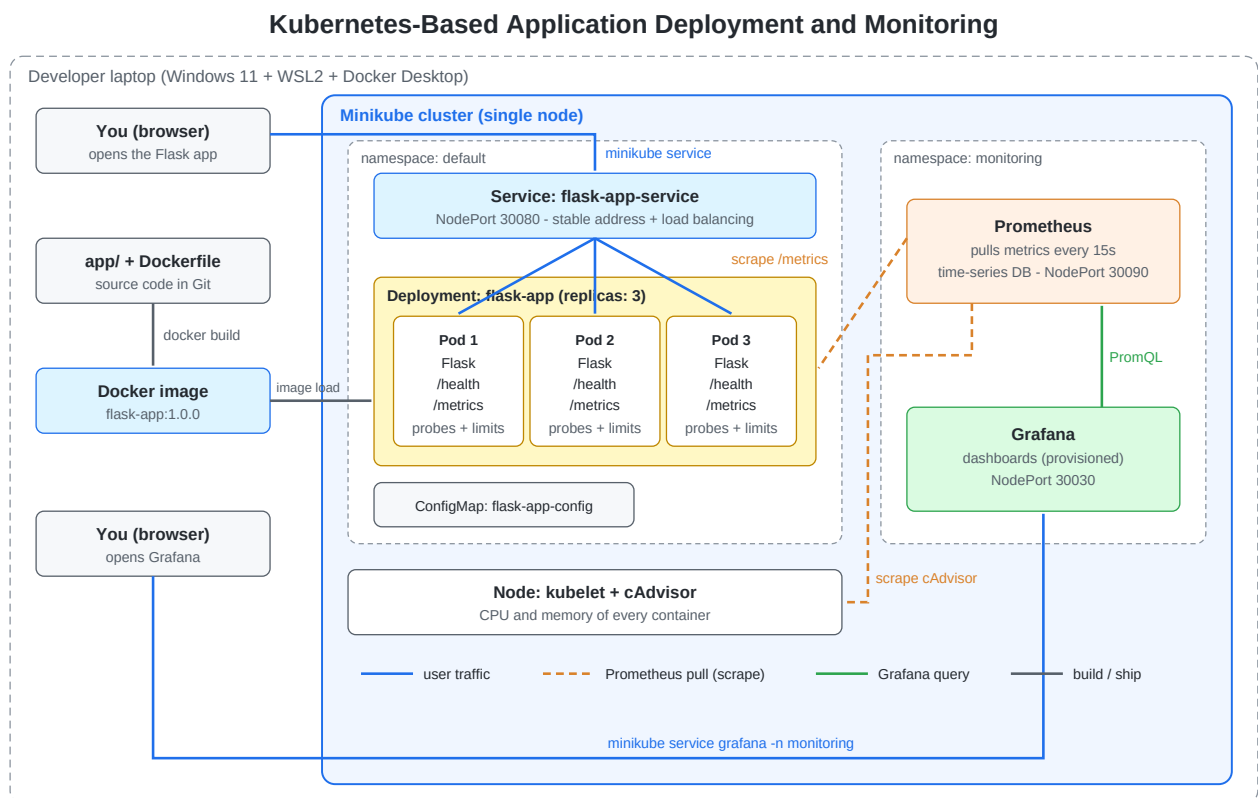


Kubernetes-Based Application Deployment and Monitoring

Docker · Minikube · Prometheus · Grafana · Python Flask

Hinglish study guide: basics se lekar interview tak, sab kuch



Radhika Agarwal

October 2026

Vishay Suchi (Contents)

Part 1 - Shuruaat: Yeh Project Kya Hai	7
Yeh guide kiske liye hai	7
Guide kaise padhein	7
Project ek line mein	7
Project ka flow (badi picture)	8
Final result kaisa dikhega	8
Is project se kya-kya seekhoge	9
Project ke rules (jo humne follow kiye)	9
Part 2 - Zero se Basics: Networking, Linux, Git, Python	10
Networking basics	10
Linux basics	11
Git aur GitHub basics	13
Python basics (jo is project mein chahiye)	13
Part 3 - Docker aur Kubernetes Basics	15
Docker basics	15
Kubernetes basics	16
Minikube	20
Part 4 - Monitoring Basics: Prometheus aur Grafana	21
Monitoring kyun?	21
Observability ke 3 pillars	21
Kya monitor karein? (Golden signals)	21
Metric types	21

Time series aur labels	22
Prometheus kaise kaam karta hai	22
PromQL (Prometheus Query Language)	23
Grafana	23
Prometheus vs Grafana (ek line mein)	24

Part 5 - Hamara Project: Har File Line by Line **25**

Folder structure	25
DRY principle: project mein kahan-kahan	25
app/config.py	26
app/app.py	26
app/gunicorn.conf.py	27
Dockerfile	27
.dockerignore	28
.gitignore aur .gitattributes	28
k8s/configmap.yaml	28
k8s/deployment.yaml (sabse important file)	29
k8s/service.yaml	30
monitoring/00-namespace.yaml	30
monitoring/10-prometheus.yaml	30
monitoring/20-grafana.yaml	32
monitoring/dashboards/flask-app.json	32
scripts/deploy.sh	33
scripts/load.sh	33

scripts/teardown.sh	34
scripts/build_pdf.py	34
tests/	34

Part 6 - Project Banate Waqt Kya-Kya Kiya Gaya 35

Step 1 - Laptop check kiya	35
Step 2 - Project folder aur Git repo	35
Step 3 - Phase 1: architecture diagram (commit 1)	35
Step 4 - Phase 2: Flask app (commit 2)	35
Step 5 - Phase 3: Docker (commit 3)	36
Step 6 - Phases 5-6: Kubernetes YAML (commit 4)	36
Step 7 - Phases 7-8: monitoring (commit 5)	36
Step 8 - Scripts (commit 6)	36
Step 9 - Phases 9-10: docs aur PDF (commit 7)	36
Step 10 - Author theek kiya	37
Step 11 - GitHub pe push	37
Step 12 - Hinglish PDF (yeh document)	37
Abhi kya baaki hai	37

Part 7 - Project Chalana: Step by Step 38

Step 1 - WSL2 install	38
Step 2 - Docker Desktop install	38
Step 3 - Project laptop pe hai na?	39
Step 4 - Tests chalao (optional, bina cluster ke)	39
Step 5 - Ek command mein sab deploy	39

Step 6 - Sab chal raha hai? Check karo	40
Step 7 - App kholo	40
Step 8 - Prometheus kholo	40
Step 9 - Grafana kholo	41
Step 10 - Load do aur graphs dekho	41
Step 11 - Screenshots lo	41
Step 12 - Kaam khatam? Band karo	41
Code badla to dobara deploy kaise karein?	41

Part 8 - Todna aur Theek Karna: Experiments aur Troubleshooting 42

Test 1 - Self-healing: pod delete karo	42
Test 2 - Liveness probe: app ko "beemar" karo	42
Test 3 - Kharab release: galat image	43
Test 4 - Memory limit: OOMKilled	43
Test 5 - CPU limit: throttling	44
Test 6 - Load mein scale karo	44
Troubleshooting ka tareeka (5 steps)	44
Error catalogue	45
Kaam ki commands	46

Part 9 - Interview Preparation 47

60-second project pitch	47
Q1. Why is Kubernetes used?	47
Q2. Difference between Pod, Deployment and Service?	47
Q3. How do replicas work?	47

Q4. How does Kubernetes handle pod failure?	48
Q5. Liveness vs readiness probe?	48
Q6. How does Prometheus collect metrics?	48
Q7. How does Grafana visualise data?	48
Q8. How do you troubleshoot a failed deployment?	49
Follow-up sawaal (chhote answers)	50
"Kya problem aayi thi?" (yeh zaroor poochenge)	50
Resume description (seedha copy karo)	51
Interview tips	51

Part 10 - Aage Kya Seekhein, Glossary aur Cheat Sheet 52

Is project ko aage kaise badhayein (ek-ek karke)	52
DevOps roadmap (internship ke liye)	52
Glossary (A to Z)	53
Cheat sheet	55
Aakhri baat	56

Appendix - Poora Source Code 57

Part 1 - Shuruaat: Yeh Project Kya Hai

Yeh guide kiske liye hai

Yeh guide aapke liye hai agar:

- Aapko Linux, Git, Docker aur Kubernetes ka **thoda-bahut** idea hai, lekin sab cheezein ek saath jodna nahi aata
- Aap DevOps internship ki tayyari kar rahe ho
- Aap chahte ho ki interview mein project ke baare mein **apne shabdon mein** confidently bol sako

Yeh guide **zero se** shuru hoti hai. Agar koi topic pehle se aata hai to skip kar do, koi baat nahi.

Guide kaise padhein

Part	Kya milega	Kab padhna hai
Part 1	Project ka overview	Sabse pehle
Part 2	Networking, Linux, Git, Python ke basics	Agar basics kamzor hain
Part 3	Docker aur Kubernetes ke basics	Zaroor padhna
Part 4	Monitoring, Prometheus, Grafana ke basics	Zaroor padhna
Part 5	Project ki har file, line by line	Code samajhne ke liye
Part 6	Is project ko banate waqt kya-kya kiya gaya	Project ki history samajhne ke liye
Part 7	Project ko laptop pe chalana, step by step	Jab chalana ho
Part 8	Cheezein jaan-bujh ke todna aur theek karna	Chalane ke baad
Part 9	Interview questions aur answers	Interview se pehle
Part 10	Aage kya seekhein, glossary, cheat sheet	Revision ke liye
Appendix	Project ka poora source code	Reference ke liye

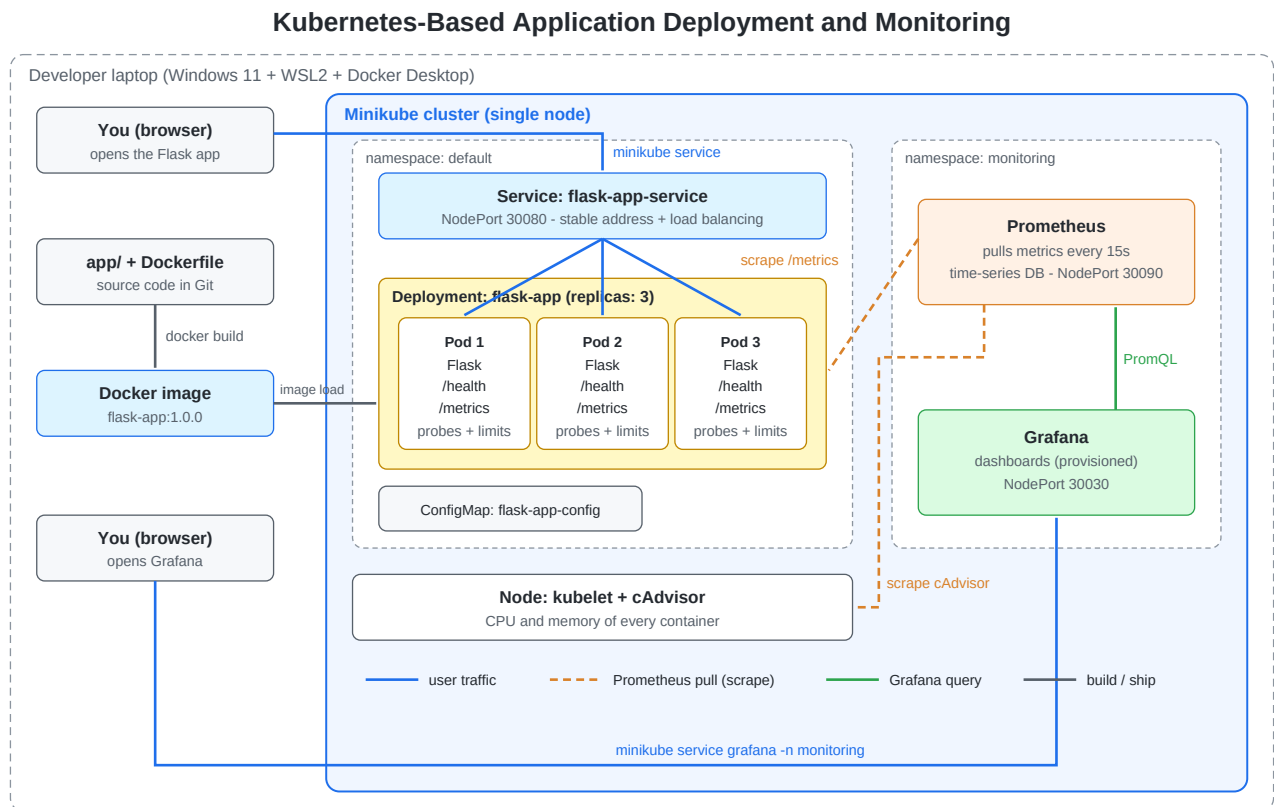
Ek zaroori baat: Har command ke saath yeh 5 cheezein di gayi hain:

- **Kya:** command kya karti hai
- **Kyun:** iski zaroorat kyun hai
- **Kahan:** kis terminal aur kis folder mein chalani hai
- **Expected:** sahi chalne pe kya dikhega
- **Error aaye to:** common galtiyan aur unka solution

Project ek line mein

Ek chhote Python Flask web app ko Docker mein pack karke, Kubernetes (Minikube) pe 3 copies mein chalaya, aur Prometheus aur Grafana se uski CPU, memory aur traffic ki monitoring ki.

Project ka flow (badi picture)



Is diagram ko step by step samjho:

1. **Code:** `app/` folder mein ek chhota Flask app hai.
2. **Docker build:** `Dockerfile` padh ke Docker ek **image** banata hai: `flask-app:1.0.0`. Image ek sealed dabba hai jisme Python, libraries aur code sab hai.
3. **Image load:** Yeh image **Minikube** ke andar bheji jaati hai. Minikube aapke laptop pe chalne wala chhota Kubernetes cluster hai.
4. **Deployment:** Kubernetes ko bola "is image ke 3 pods chalao". Pod matlab app ki ek running copy.
5. **Service:** 3 pods ke aage ek fixed address hai: `flask-app-service`. Browser se request aati hai to Service use kisi ek pod ko bhej deta hai.
6. **Prometheus:** Har 15 second mein jaake poochta hai, "CPU kitna use hua? Memory kitni? Kitni requests aayi?" Aur yeh sab numbers store karta hai.
7. **Grafana:** Prometheus se numbers leke graphs banata hai. Aap browser mein dashboard dekhte ho.

Final result kaisa dikhega

Jab project chal raha hoga:

- Browser mein app ka homepage dikhega, likha hoga "Served by pod flask-app-xxxxx". Refresh karne pe kabhi-kabhi pod ka naam badlega (load balancing).
- `kubectl get pods` chalane pe 3 pods **Running** dikhenge.
- Grafana mein dashboard dikhega: CPU graph, memory graph, requests per second, response time, aur "healthy pods = 3".

- Ek pod delete karoge to kuch second mein naya pod apne aap aa jaayega. Isko **self-healing** kehte hain.

Is project se kya-kya seekhoge

Skill	Project mein kahan
Containerization	Dockerfile, image, layers
Container orchestration	Deployment, ReplicaSet, Service
High availability	3 replicas, self-healing
Health checks	Readiness aur liveness probes
Resource management	Requests, limits, OOMKilled, throttling
Zero-downtime deployment	Rolling update, rollback
Configuration management	ConfigMap, environment variables
Security basics	Non-root container, RBAC
Monitoring	Prometheus, PromQL, service discovery
Visualization	Grafana dashboard, provisioning
Troubleshooting	describe, logs, events, rollout undo
Automation	Bash scripts, automated tests
Version control	Git, GitHub, commits

Project ke rules (jo humne follow kiye)

1. **Simple rakho.** Helm, Terraform, cloud jaisa kuch nahi. Pehle basics pakke karo.
2. **Har line samajh aani chahiye.** Interview mein koi bhi line pooch sakte hain.
3. **DRY (Don't Repeat Yourself):** har jaankari **sirf ek jagah** likhi ho. Isko Part 5 mein detail mein samjhaya hai.
4. **Dikhawe ke liye koi technology nahi.** Jo zaroori hai, bas wahi.

Part 2 - Zero se Basics: Networking, Linux, Git, Python

Is part mein woh basic cheezein hain jinke bina aage ka kuch samajh nahi aayega. Agar pehle se aata hai to jaldi se padh lo, revision ho jaayega.

Networking basics

Server aur client

- **Client:** jo request bhejta hai, jaise aapka browser ya `curl` command.
- **Server:** jo request sunta hai aur jawab deta hai, jaise hamara Flask app.

Ek hi laptop pe dono ho sakte hain. Jab aap `localhost:5000` kholte ho, to browser (client) aur Flask (server) dono aapke laptop pe hote hain.

IP address aur port

- **IP address:** computer ka address, jaise ghar ka address. Example: `192.168.1.5`, `10.244.0.12`.
- **Port:** us computer ke andar kaun sa program, jaise building ke andar flat number. Example: `5000`, `80`, `9090`.

Poora address = IP + port, jaise `10.244.0.12:5000`.

`localhost` / `127.0.0.1` ka matlab hai "yahi computer".

`0.0.0.0` pe bind karna ka matlab hai "har network interface pe suno". Container ke andar app ko `0.0.0.0` pe sunna padta hai, warna container ke bahar se koi connect nahi kar paayega. Isliye `unicorn.conf.py` mein `bind = "0.0.0.0:5000"` likha hai.

Project mein use hue ports

Port	Kaun	Kahan
5000	Flask app	Container ke andar
80	flask-app-service	Cluster ke andar
30080	flask-app-service (NodePort)	Minikube node pe
9090 / 30090	Prometheus	Cluster ke andar / node pe
3000 / 30030	Grafana	Cluster ke andar / node pe

HTTP request aur response

Browser jab koi page kholta hai, to ek **HTTP request** bhejta hai:

```
GET /health HTTP/1.1
Host: localhost:5000
```

Server **HTTP response** wapas bhejta hai:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status": "ok", "pod": "flask-app-7d9f-abc12"}
```

HTTP methods:

Method	Matlab	Project mein
GET	Data lao	/, /health, /metrics, /work
POST	Kuch karo / data bhejo	/fail

Status codes: (yeh interview mein bhi poochte hain)

Code	Matlab	Project mein kab
200	OK, sab theek	/health jab app healthy hai
404	Page nahi mila	Galat URL
500	Server mein error	Code mein bug
503	Service unavailable (abhi kaam nahi kar sakta)	/health jab /fail call ho chuka ho

Kubernetes ke probes ke liye **200-399 = pass**, aur baaki sab **fail**.

JSON

JSON data likhne ka format hai, jaise Python dictionary:

```
{"status": "ok", "pod": "flask-app-abc"}
```

APIs aksar JSON mein jawab dete hain. Hamara /health bhi JSON deta hai.

DNS

DNS naam ko IP mein badalta hai, jaise phone ki contact list. `google.com` se `142.250.x.x` milta hai.

Kubernetes ke andar bhi DNS hota hai: `flask-app-service` naam likhne se Service ka IP mil jaata hai. Isliye Grafana ki config mein `http://prometheus:9090` likha hai, IP nahi.

Linux basics

Terminal aur shell

Terminal woh window hai jahan commands likhte ho. **Shell** woh program hai jo commands samajhta hai: bash, PowerShell, zsh.

Windows pe hum **Git Bash** use karte hain, jo Git ke saath aata hai aur Linux jaisi bash commands chalata hai. Isse hamari `.sh` scripts Windows pe bhi chal jaati hain.

Kuch zaroori commands

Command	Kya karti hai
<code>pwd</code>	Abhi kis folder mein ho
<code>ls</code>	Folder mein kya hai
<code>cd folder</code>	Folder mein jao
<code>mkdir -p a/b</code>	Folder banao (-p = parent bhi bana do)
<code>cat file</code>	File ka content dikhao
<code>grep word file</code>	File mein word dhoondho
<code>curl url</code>	URL pe request bhejo, jawab dikhao
<code>echo \$VAR</code>	Variable ki value dikhao
<code>cmd1 cmd2</code>	Pipe: cmd1 ka output cmd2 ko do
<code>chmod +x file</code>	File ko executable banao

Process

Jo program chal raha hai, use **process** kehte hain. Har process ka ek number hota hai (**PID**). Container ke andar main process ka PID 1 hota hai. Hamare case mein woh unicorn hai.

Environment variables

Ye process ko di gayi **settings** hain, **NAAM=value** format mein:

```
export PORT=5050
echo $PORT          # 5050
```

Hamara app `config.py` mein `os.getenv("PORT", "5000")` se padhta hai: "PORT variable ho to woh lo, warna 5000."

Kyun? Code badle bina behaviour badal sakte ho. Kubernetes isi tarah ConfigMap se settings deta hai.

Signals aur exit codes

Signal ek process ko bheja gaya message hota hai:

Signal	Number	Matlab
SIGTERM	15	"Please band ho jao" (process saaf-safai karke band hota hai)
SIGKILL	9	"Turant maro" (process kuch nahi kar sakta)

Exit code process band hone pe deta hai: **0** = success, aur baaki sab = koi problem.

Signal se mare process ka exit code **128 + signal number** hota hai. Isliye **OOMKilled = 137** (128 + 9). Yeh interview mein bahut poocha jaata hai.

Kernel, namespaces, cgroups

Yeh container ki neev hain.

- **Kernel:** Linux ka core. Hardware, memory aur processes sambhalta hai.
- **Namespaces:** process ko lagta hai woh akela hai, uska apna hostname, network aur process list hai. Isse **isolation** milta hai.
- **cgroups (control groups):** process kitna CPU/memory use kar sakta hai, uski **limit**. Kubernetes ke resource limits isi se lagte hain.

Container = normal Linux process + namespaces (alag duniya) + cgroups (limits). Yeh koi alag VM nahi hai.

Windows pe kernel Linux ka nahi hai, isliye **WSL2** chahiye. Woh Windows ke andar ek asli Linux kernel chalata hai.

Git aur GitHub basics

Term	Matlab
Repository (repo)	Project folder jiski history Git rakhta hai (.git folder)
Commit	Ek snapshot / save point, message ke saath
Branch	Kaam ki alag line. Humari main branch main hai
Remote	GitHub pe repo ki copy (origin)
Push	Local commits GitHub pe bhejna
Pull	GitHub se naye commits laana
.gitignore	Kaun si files Git mein nahi jaani chahiye (.venv , cache)
.gitattributes	Files ko kaise treat karna hai (line endings)

Basic workflow

```
git status          # kya badla hai
git add file        # commit ke liye select karo
git commit -m "message" # save point banao
git push           # GitHub pe bhejo
git log --oneline  # history dekho
```

Author identity

Har commit pe likha hota hai ki kisne kiya: **user.name** aur **user.email**. GitHub **email** se pehchanta hai ki commit kis account ka hai. Is project mein repo-level config set kiya gaya:

```
git config user.name "Radhika Agarwal"
git config user.email "theradhika2410@gmail.com"
```

--global ke bina yeh setting sirf **isi repo** pe lagti hai.

Line endings (CRLF vs LF)

Windows har line ke end mein **\r\n** (CRLF) lagata hai, aur Linux sirf **\n** (LF). Agar shell script CRLF ke saath Linux container mein chale, to error aata hai: **\$'\r': command not found**.

Isliye **.gitattributes** mein *** text=auto eol=lf** likha hai, jisse Git hamesha LF rakhta hai.

Achha commit message

```
Add readiness probe to flask-app deployment    <- chhoti summary, command ki tarah
Pods were receiving traffic before Flask had started.  <- kyun kiya
```

Python basics (jo is project mein chahiye)

Virtual environment (venv)

Har project ki libraries alag rakhne ka tareeka:

```
python -m venv .venv          # .venv folder banao
source .venv/Scripts/activate  # activate (Linux/Mac: .venv/bin/activate)
pip install -r requirements.txt # libraries install
```

Kyun? Ek project ko Flask 3 chahiye aur doosre ko Flask 2, to dono ek saath chal sakte hain.

pip aur requirements.txt

pip Python ka package installer hai. **requirements.txt** mein libraries **exact version** ke saath likhi hain:

```
Flask==3.1.0
prometheus-client==0.21.1
gunicorn==23.0.0
```

Version pin kyun? Aaj aur 6 mahine baad bhi same version install ho. Isse "kal tak chal raha tha" wali problem nahi aati.

Flask kaise kaam karta hai

```
from flask import Flask
app = Flask(__name__)

@app.get("/health")          # decorator: "GET /health aaye to neeche wala function chalao"
def health():
    return {"status": "ok"}
```

- **@app.get(...)** ko **decorator** kehte hain. Yeh URL ko function se jodta hai.
- Function jo return karta hai, woh response ban jaata hai.

Flask dev server vs gunicorn

	Flask dev server (app.run())	Gunicorn
Kis liye	Development, debugging	Production
Speed / stability	Kam	Zyada
Auto-reload	Haan	Nahi
Windows pe	Chalta hai	Nahi chalta (sirf Linux/Mac)

Isliye laptop pe testing **python app/app.py** se hoti hai, aur container mein gunicorn chalta hai.

Workers aur threads: Worker ek alag process hai, aur thread ek process ke andar kaam karne wali line. Humne **workers = 1, threads = 4** rakha. Kubernetes mein scale **pods badha ke** karte hain, workers badha ke nahi.

pytest

Python ka testing tool hai. **test_** se shuru hone wale functions apne aap chalaata hai:

```
def test_health_is_ok(client):
    response = client.get("/health")
    assert response.status_code == 200    # agar galat hua to test FAIL
```

assert ka matlab hai "yeh sach hona chahiye, warna fail".

Part 3 - Docker aur Kubernetes Basics

Docker basics

Problem kya thi?

Aapne laptop pe app banaya, jisme Python 3.12 aur Flask 3.1 tha. Server pe Python 3.8 hai aur Flask hai hi nahi, to app nahi chala. Yahi hai **"It works on my machine"** problem.

Docker ka solution: app ke saath uska **poora environment** (OS files, Python, libraries, code) ek package mein band kar do. Yeh package har jagah same chalega.

Virtual Machine vs Container

	Virtual Machine (VM)	Container
Kya hai	Poora alag computer (apna OS kernel)	Ek isolated process (host ka kernel share karta hai)
Size	GBs	MBs
Start time	Minutes	Seconds
Isolation	Bahut strong	Achha (namespaces + cgroups)
Example	VirtualBox, Hyper-V	Docker

Image vs Container

Image	Container
Blueprint / recipe / class	Running cheez / dish / object
Read-only	Running process
<code>docker build</code> se banti hai	<code>docker run</code> se chalta hai
Ek image	Uske kai containers ban sakte hain

Analogy: Image = cake ki recipe, container = bana hua cake. Ek recipe se 10 cake ban sakte hain.

Layers

Dockerfile ki har instruction ek **layer** banati hai. Layers ek ke upar ek rakhi jaati hain.

Caching: Agar kisi layer ke inputs nahi badle, to Docker purani layer reuse karta hai. Isliye hum `requirements.txt` pehle copy karte hain aur code baad mein. Code badalne pe `pip install` wali layer cache se aa jaati hai, aur build seconds mein ho jaata hai.

Registry

Images store karne ki jagah, jaise GitHub code ke liye hota hai. Example: **Docker Hub** (`python:3.12-slim` wahan se aata hai), GitHub Container Registry, AWS ECR.

Is project mein hum image **registry pe push nahi karte**. Seedha Minikube mein load karte hain (`minikube image load`), jo local learning ke liye simple hai.

Image naam ka format

```
flask-app:1.0.0
|           |
naam       tag (version)
```

Tag na do to **latest** maan liya jaata hai. ****latest avoid karo****, kyunki pata nahi chalta kaun sa version chal raha hai. Humne **1.0.0** rakha.

Zaroori Docker commands

Command	Kya karti hai
<code>docker build -t naam:tag .</code>	Current folder ke Dockerfile se image banao
<code>docker images</code>	Saari images dikhao
<code>docker run -d -p 5000:5000 --name x image</code>	Container chalao (background, port map)
<code>docker ps</code>	Chal rahe containers
<code>docker logs x</code>	Container ke logs
<code>docker exec -it x sh</code>	Container ke andar shell kholo
<code>docker rm -f x</code>	Container band karke hatao

****Port mapping -p 5000:5000:**** `laptop_port:container_port`. Container ka apna network hai, aur bina mapping ke laptop se andar nahi pahunch sakte.

Docker Desktop aur WSL2

Windows pe Docker Desktop ek chhota Linux environment (WSL2) chalata hai aur containers wahan chalte hain. Isliye Windows pe pehle **WSL2** chahiye.

Kubernetes basics

Kubernetes kyun?

Ek container chalana easy hai: `docker run`. Lekin production mein sawaal aate hain:

- Container crash ho gaya to **restart** kaun karega?
- Traffic badh gaya to 10 copies kaise chalayein, aur unme **traffic kaise baantein**?
- Naya version **bina downtime** kaise daalein? Kharab nikla to **wapas** kaise jaayein?
- 50 servers hain to container **kis server pe** chalega?

Kubernetes (K8s) yeh sab **automatic** karta hai. Isko **container orchestrator** kehte hain, jaise orchestra ka conductor saare musicians ko sambhalta hai.

Declarative vs Imperative

Imperative ("kaise karo")	Declarative ("kya chahiye")
"Container 1 chalao, phir 2, phir 3"	"Mujhe 3 containers chahiye"
Aap har step batate ho	Aap final state batate ho, K8s khud karta hai
<code>docker run x 3</code>	<code>replicas: 3</code> in YAML

Kubernetes **declarative** hai. Aap YAML mein **desired state** likhte ho, aur Kubernetes lagatar **actual state** ko desired state jaisa banata rehta hai. Isko **reconciliation loop** kehte hain, aur yahi Kubernetes ka dil hai.

Cluster architecture

Cluster = machines (nodes) ka group jo milkar Kubernetes chalte hain.

Control plane (dimaag):

Component	Kaam
API server	Sab ka entry gate. kubectl isi se baat karta hai
etcd	Database jahan cluster ki saari state store hoti hai
Scheduler	Naya pod kis node pe chalega, yeh decide karta hai
Controller manager	Reconciliation loops chalata hai (ReplicaSet controller etc.)

Worker node (haath-pair):

Component	Kaam
kubelet	Node pe pods chalata hai, probes check karta hai, container restart karta hai
Container runtime	Asli mein containers chalata hai (containerd / Docker)
kube-proxy	Service ka networking (traffic sahi pod tak pahunchana)
cAdvisor	kubelet ke andar. Har container ka CPU/memory naapta hai, aur Prometheus yahin se data leta hai

Minikube mein yeh sab ek hi node pe hai: control plane bhi wahi, worker bhi wahi.

kubectl

Kubernetes se baat karne ka command-line tool:

```
kubectl get pods          # list
kubectl describe pod <naam> # detail + events
kubectl logs <naam>       # logs
kubectl apply -f file.yaml # YAML lagao (create ya update)
kubectl delete -f file.yaml # YAML wali cheezein hatao
```

kubectl ko kaise pata ki kis cluster se baat karni hai? `~/.kube/config` file se, jise **kubeconfig** kehte hain. **minikube start** yeh apne aap set kar deta hai.

YAML ka structure

Har Kubernetes object ke 4 hisse hote hain:

```
apiVersion: apps/v1      # kaun sa API group/version
kind: Deployment          # kya cheez hai
metadata:                 # naam, labels, namespace
  name: flask-app
spec:                     # DESIRED STATE - kya chahiye
  replicas: 3
```

YAML rules: indentation **spaces** se hoti hai (tab nahi), **key: value** likhte hain, list ke liye - **item**. Ek file mein kai objects `---` se alag hote hain.

Labels aur selectors

Label = object pe chipka hua tag, jaise **app: flask-app**. **Selector** = "jin objects pe yeh label hai, woh sab".

Kubernetes mein cheezein **naam se nahi, labels se** judti hain:

- Deployment apne pods ko label se pehchanta hai
- Service apne pods ko label se dhoondhta hai
- Hamari Prometheus config bhi pods ko annotation/label se dhoondhti hai

Sabse common galti: selector aur label match nahi kar rahe. Tab Service ke peeche koi pod nahi hota. Hamare **test_manifests.py** tests yahi check karte hain.

Namespace

Cluster ke andar alag-alag "kamre". Same naam ki cheez alag namespaces mein ho sakti hai. Humne app ko **default** mein rakha aur monitoring ko **monitoring** namespace mein.

Pod

- Kubernetes ki **sabse chhoti unit**
- Ek ya zyada containers, jo **ek IP** share karte hain
- **Temporary hai**. Pod mara to wahi pod wapas nahi aata, **naya pod** (naye naam aur naye IP ke saath) banta hai
- Hum seedha pod nahi banate, Deployment banata hai

ReplicaSet

"Is label wale exactly N pods hone chahiye." Kam hue to banao, zyada hue to hatao. Deployment isko khud banata hai, aap seedha use nahi karte.

Deployment

ReplicaSet ke upar ek manager:

- Replicas maintain karta hai (ReplicaSet ke through)
- **Rolling update**: naya version dheere-dheere laata hai, purana dheere-dheere hatata hai
- **Rollback**: **kubectl rollout undo** se pichla version wapas laata hai
- History rakhta hai (revisions)

Rolling update ke andar kya hota hai? Image badalne pe Deployment ek **naya ReplicaSet** banata hai. Naya wala 0 se 3 tak jaata hai aur purana 3 se 0 tak, ek-ek karke. Hamari settings:

- **maxSurge: 1**: update ke time max 1 extra pod (yaani 4 tak)
- **maxUnavailable: 0**: kabhi bhi 3 se kam **ready** pods nahi

Isliye agar naya version kharab nikle, to purane pods chalte rehte hain. Site down nahi hoti.

Service

Problem: pods aate-jaate rehte hain aur unke IP badalte rehte hain. Client kis IP pe jaaye?

Solution: Service. Ek **fixed IP + DNS naam**, jo label se pods dhoondhta hai aur **sirf ready pods** ko traffic bhejta hai (load balancing).

Service type	Kahan se access	Kab use karein
ClusterIP (default)	Sirf cluster ke andar se	Internal services (database etc.)
NodePort	Node ke IP + port 30000-32767 se	Local testing, Minikube
LoadBalancer	Cloud ka load balancer, public IP	Cloud (AWS, GCP, Azure)

Humne **NodePort** liya, kyunki Minikube pe bahar se access karne ka yeh sabse simple tareeka hai.

Endpoints: Service ke peeche abhi kaun-kaun se pod IPs hain. **kubectl get endpoints flask-app-service** chalao. Khaali (**<none>**) aaye to selector galat hai ya koi pod ready nahi hai.

Service ke 3 ports:

```
port: 80          # cluster ke andar: http://flask-app-service:80
targetPort: http # container ka port (naam "http" = 5000)
nodePort: 30080  # node pe bahar se
```

ConfigMap aur Secret

ConfigMap	Secret
Normal settings (APP_ENV=minikube)	Sensitive cheezein (password, API key)
Plain text	Base64 encoded (encryption nahi, sirf encoding!)

Fayda: image ek hi rehti hai, settings environment ke hisaab se badalti hain. Dev, test aur prod sab mein same image chalti hai.

Probes (health checks)

kubelet pod ko baar-baar check karta hai:

	Readiness probe	Liveness probe	Startup probe
Sawaal	Traffic le sakta hai?	Zinda hai ya atak gaya?	Start ho gaya?
Fail hone pe	Service se hata do	Container restart	Baaki probes rok ke rakho
Humne use kiya?	Haan	Haan	Nahi (Flask 1 sec mein start hota hai)

Probe ke types: **httpGet** (URL check, jo humne use kiya), **tcpSocket** (port khula hai ya nahi), **exec** (container ke andar command chhalao).

Probe settings:

Setting	Matlab
initialDelaySeconds	Container start hone ke kitni der baad pehla check
periodSeconds	Kitni-kitni der mein check
failureThreshold	Kitni baar lagatar fail hone pe action
timeoutSeconds	Kitni der mein jawab na aaye to fail (default 1s)

Resources: requests aur limits

```
resources:
  requests: {cpu: 100m, memory: 64Mi}
  limits:   {cpu: 250m, memory: 128Mi}
```

- **CPU units:** **1** = 1 core, **100m** = 100 millicores = 0.1 core
- **Memory units:** **Mi** = mebibyte (1024 x 1024 bytes), **Gi** = gibibyte

	Requests	Limits
Matlab	Minimum guarantee	Maximum hadd
Kaun dekhta hai	Scheduler (kis node pe jagah hai)	kubelet / kernel (cgroups)
CPU zyada	-	Throttling (slow ho jaata hai, marta nahi)
Memory zyada	-	OOMKilled (exit code 137)

CPU aur memory mein farq kyun? CPU "compressible" hai, use thoda kam do to process slow chalega. Memory "incompressible" hai, jo le li woh wapas nahi le sakte, to process ko maarna padta hai.

QoS classes (memory kam pade to kaun pehle maara jaayega):

Class	Kab	Priority
Guaranteed	requests = limits (har container mein)	Sabse safe
Burstable	requests < limits	Beech mein (hamare pods yahi hain)
BestEffort	Koi requests/limits nahi	Sabse pehle maara jaayega

RBAC (Role-Based Access Control)

Kaun kya kar sakta hai? Kubernetes mein 4 cheezein hoti hain:

Object	Kaam
ServiceAccount	Pod ki identity ("main Prometheus hoon")
Role / ClusterRole	Permissions ki list ("pods dekh sakte ho")
RoleBinding / ClusterRoleBinding	Identity ko permission dena

Role ek namespace ke liye hota hai aur ClusterRole poore cluster ke liye. Prometheus ko poore cluster ke pods aur nodes dekhne hain, isliye **ClusterRole**. Sirf **get**, **list**, **watch** diye gaye hain (read-only). Isko **least privilege** kehte hain: jitna zaroori hai, bas utna.

Minikube

Laptop pe chhota, asli Kubernetes cluster.

Concept	Matlab
Driver	Minikube node kahan chalega: docker (container), virtualbox (VM), hyperv. Humne docker liya
Image store	Minikube ka apna image store hai, laptop ke Docker se alag. Isliye minikube image load karna padta hai
minikube service	Docker driver pe Windows/Mac mein node ka IP seedha nahi khulta, to yeh ek tunnel banata hai. Terminal khula rakhna padta hai
Addons	Extra features ek command se: metrics-server , dashboard , ingress

Minikube vs alternatives: kind (Kubernetes in Docker), k3s (halka Kubernetes), Docker Desktop ka built-in Kubernetes. Sab learning ke liye theek hain. Minikube sabse popular aur beginner-friendly hai.

Minikube vs production (EKS/GKE/AKS): production mein kai nodes hote hain, cloud load balancer hota hai, storage cloud ka hota hai, aur control plane cloud provider sambhalta hai. Lekin YAML aur concepts **wahi** rehte hain. Isliye Minikube pe seekha hua seedha kaam aata hai.

Part 4 - Monitoring Basics: Prometheus aur Grafana

Monitoring kyun?

App chal raha hai, lekin:

- Kya woh **slow** ho raha hai?
- Memory **dheere-dheere badh** rahi hai (memory leak)?
- Kitne log use kar rahe hain?
- Raat 3 baje crash hua to **kyun**?

Bina monitoring ke yeh sab andaaza hai. Monitoring se **data** milta hai.

Observability ke 3 pillars

Pillar	Kya hai	Example	Tool
Metrics	Numbers, time ke saath	CPU = 0.12 cores at 10:05	Prometheus (is project mein)
Logs	Text events	<code>GET /health 200</code>	<code>kubectl logs</code> , Loki, ELK
Traces	Ek request ka poora safar, services ke beech	Request ne 3 services mein 200ms liye	Jaeger, Tempo

Is project mein focus **metrics** pe hai. Logs `kubectl logs` se dekh lete hain.

Kya monitor karein? (Golden signals)

Google SRE book ke **4 golden signals**:

Signal	Matlab	Hamara dashboard panel
Latency	Response kitni der mein aaya	95th percentile response time
Traffic	Kitni requests aa rahi hain	Requests per second
Errors	Kitni requests fail hui	<code>status</code> label (5xx count)
Saturation	Resources kitne bhare hain	CPU aur memory panels

Metric types

Type	Kya hai	Example	Kaise padhein
Counter	Sirf badhta hai (restart pe 0)	<code>flask_http_requests_total</code>	Hamesha <code>rate()</code> ke saath
Gauge	Upar-neeche ho sakta hai	<code>container_memory_working_set_bytes</code>	Seedha value
Histogram	Values ko buckets mein ginta hai	<code>flask_http_request_duration_seconds</code>	<code>histogram_quantile()</code> se percentile
Summary	Histogram jaisa, client pe calculate	(use nahi kiya)	-

****Counter pe `rate()` kyun?**** Counter ki value "kul 50,000 requests" batati hai, jo kaam ki nahi. `rate()` batata hai "abhi 12 requests per second", jo kaam ki hai.

Time series aur labels

Prometheus mein har data point aisa dikhta hai:

```
flask_http_requests_total{endpoint="/health", method="GET", status="200", pod="flask-app-abc"} 42 @10:05:00
      naam                                labels
      value time
```

Naam + labels ka har unique combination = ek time series.

Labels se filter aur group karte hain: "sirf **/work** ki requests", "har pod ka alag".

Cardinality (zaroori concept)

Agar label mein **unlimited values** aa sakti hain (jaise user ID ya poora URL), to lakhon time series ban jaati hain aur Prometheus ki memory bhar jaati hai. Isko **high cardinality** problem kehte hain.

Isliye hamare app mein **endpoint** label mein **route pattern** (**/work**) jaata hai, poora URL (**/work?ms=123**) nahi.

Prometheus kaise kaam karta hai

Pull model

```
har 15 second
Prometheus ----- GET /metrics -----> App pod
      <----- text metrics -----
      (store karo time ke saath)
```

Prometheus **khud jaake** data leta hai, aur app kuch push nahi karti. Isko **scraping** kehte hain.

Pull ke fayde:

- Target down hai to Prometheus ko turant pata chalta hai (**up = 0**)
- App ko Prometheus ka address jaanne ki zaroorat nahi
- Browser mein **/metrics** khol ke khud dekh sakte ho ki kya bhej rahe ho

(Push model bhi hota hai, jaise Pushgateway short jobs ke liye. Lekin default pull hai.)

/metrics ka format

```
# HELP flask_http_requests_total Total HTTP requests
# TYPE flask_http_requests_total counter
flask_http_requests_total{endpoint="/health",method="GET",status="200"} 42.0
```

Python ki **prometheus_client** library yeh format apne aap banati hai.

Targets aur jobs

- **Target:** ek address jahan se scrape karna hai (**10.244.0.5:5000**)
- **Job:** ek jaise targets ka group (**app-pods**)

Service discovery

Pods ke IP badalte rehte hain, to targets ki list hath se nahi likh sakte. Prometheus **Kubernetes API se poochta hai:** "saare pods ki list do." Naya pod aaya to apne aap target ban jaata hai. Iske liye RBAC permission chahiye (Part 3).

Relabeling

Discovery se **saare** pods milte hain, aur relabeling rules unhe filter aur rename karte hain:

- **action:** **keep** + annotation **prometheus.io/scrape: "true"** ka matlab hai sirf woh pods jinpe yeh annotation hai
- Port ka naam **http** hai to sirf woh port rakho
- Pod ka naam **pod** label mein daalo, taaki graph mein dikhe

cAdvisor

Container Advisor, jo kubelet mein built-in hai. Har container ka CPU, memory, network aur disk naapta hai. Hamare app ko CPU/memory metrics khud nahi dene padte, **cAdvisor deta hai**.

Prometheus storage

Prometheus ka apna **time-series database (TSDB)** hai, disk pe. Humne **emptyDir** volume diya, jo pod restart pe saaf ho jaata hai. Learning ke liye theek hai, production mein PersistentVolume chahiye.

PromQL (Prometheus Query Language)

Query	Matlab
<code>up</code>	Har target: 1 = theek, 0 = down
<code>flask_http_requests_total</code>	Raw counter values
<code>rate(flask_http_requests_total[1m])</code>	Pichle 1 minute mein per-second rate
<code>sum(rate(flask_http_requests_total[1m]))</code>	Saare pods mila ke total rate
<code>sum by (pod) (rate(...[1m]))</code>	Har pod ka alag total
<code>container_memory_working_set_bytes{container="flask-app"}</code>	<code>{}</code> se label filter
<code>histogram_quantile(0.95, sum by (le) (rate(..._bucket[5m])))</code>	95% requests isse tez the (p95 latency)

****[1m] = range: pichle 1 minute ka data. sum by (pod) = pod ke hisaab se group karke jod do.** p95 = 0.2s** ka matlab hai 95% requests 0.2 second se kam mein पूरी hui. Average ki jagah percentile isliye dekhte hain kyunki average slow requests ko chhupa deta hai.

CPU ka formula samjho

```
rate(container_cpu_usage_seconds_total[1m])
```

container_cpu_usage_seconds_total = container ne ab tak kul kitne **CPU-seconds** use kiye. **rate()** = har second mein kitne CPU-seconds, yaani **kitne cores**. Value **0.25** = 0.25 core = 250m. Yeh hamari limit ke barabar hai.

Memory: working set kyun?

container_memory_working_set_bytes = woh memory jo kernel aasani se wapas nahi le sakta. **OOM killer yahi dekhta hai**, isliye limit se compare isi ko karte hain.

Grafana

Grafana kya hai?

Visualization tool. **Khud koi data store nahi karta**. Data sources (Prometheus, MySQL, Loki...) se query karke graphs banata hai.

Term	Matlab
Data source	Data kahan se aayega (hamara: Prometheus)
Dashboard	Panels ka collection
Panel	Ek graph/number. Ek query + visualization type
Visualization	Time series (line graph), Stat (bada number), Gauge, Table
Provisioning	Data source aur dashboard files se apne aap bana dena

Provisioning kyun?

Hath se click karke dashboard banaya, Grafana pod restart hua, aur sab gaya (kyunki storage `emptyDir` hai). **Provisioning** se:

- Data source aur dashboard YAML/JSON files mein hain, **Git mein**
- Grafana start hote hi apne aap load
- Kitni baar bhi reinstall karo, same dashboard

Isi soch ko **"Infrastructure as Code"** kehte hain: config ko code ki tarah rakho.

`$_rate_interval`

Grafana ka special variable. Scrape interval aur zoom level dekh ke `rate()` ke liye sahi window chunta hai. Window bahut chhoti ho to graph khaali dikhega, isliye yeh safe option hai.

Prometheus vs Grafana (ek line mein)

Prometheus collect aur store karta hai. Grafana dikhata hai.

Prometheus ka apna basic graph UI bhi hai (Graph tab), lekin dashboards ke liye Grafana kahin behtar hai.

Part 5 - Hamara Project: Har File Line by Line

Poora code Appendix mein hai. Yahan har file ka **matlab** samjhaya gaya hai: kya hai, kyun hai, aur kaise kaam karta hai.

Folder structure

```
k8s-flask-monitoring/
  app/
    app.py           Flask application
    config.py        routes + Prometheus metrics
    config.py        saari settings ek jagah (DRY)
    gunicorn.conf.py production server settings
    requirements.txt libraries + versions
  k8s/
    configmap.yaml   app ke Kubernetes files
    deployment.yaml  replicas, probes, resources
    service.yaml     NodePort 30080
  monitoring/
    00-namespaces.yaml Prometheus + Grafana (number = order)
    10-prometheus.yaml
    20-grafana.yaml
    dashboards/flask-app.json
  scripts/
    automation
    deploy.sh load.sh teardown.sh build_pdf.py
  tests/
    automated tests
  docs/
    guides, diagram, PDF, screenshots
  Dockerfile .dockerignore .gitignore .gitattributes
  requirements-dev.txt pytest.ini README.md
```

DRY principle: project mein kahan-kahan

DRY = Don't Repeat Yourself. Har jaankari **sirf ek jagah** likho. Do jagah likhoge to ek din ek jagah badloge aur doosri bhool jaaoge, aur bug aayega.

Jaankari	Sirf yahan likhi hai	Kaun-kaun use karta hai
Port, app name, version ke defaults	<code>app/config.py</code>	<code>app.py</code> , <code>gunicorn.conf.py</code>
Image tag <code>1.0.0</code>	<code>k8s/deployment.yaml</code> ki <code>image:</code> line	<code>deploy.sh</code> wahi se padhta hai aur <code>docker build</code> ko deta hai
Port number 5000	<code>containerPort</code> jiska naam <code>http</code> hai	Probes, Service <code>targetPort</code> , Prometheus sab <code>http</code> naam use karte hain
Health check	YAML anchor <code>&health-check</code>	Readiness aur liveness dono
Grafana dashboard	<code>monitoring/dashboards/flask-app.json</code>	<code>deploy.sh</code> usse ConfigMap banata hai, copy nahi karta
Datasource uid <code>prometheus</code>	<code>20-grafana.yaml</code>	Dashboard JSON; test check karta hai ki dono match hon
Documentation	<code>docs/*.md</code>	README links deta hai, PDF unhi se banta hai

DRY ki limit bhi samjho: Kubernetes mein label `app: flask-app` Deployment ke selector aur template dono mein likhna **zaroori** hai, kyunki Kubernetes aisa hi maangta hai. Aisi jagah humne **test** likh diya jo check karta hai ki dono match karein. Jahan repeat karna pade, wahan test se safety lo.

app/config.py

```
@dataclass(frozen=True)
class Settings:
    app_name: str = field(default_factory=lambda: _env("APP_NAME", "flask-app"))
    app_version: str = field(default_factory=lambda: _env("APP_VERSION", "dev"))
    app_env: str = field(default_factory=lambda: _env("APP_ENV", "local"))
    port: int = field(default_factory=lambda: int(_env("PORT", "5000")))
    hostname: str = field(default_factory=lambda: _env("HOSTNAME", "localhost"))

settings = Settings()
```

Line	Matlab
<code>@dataclass(frozen=True)</code>	Ek simple class jiske fields baad mein badle nahi ja sakte (frozen)
<code>_env("APP_NAME", "flask-app")</code>	Environment variable APP_NAME ho to woh, warna "flask-app"
<code>int(...)</code>	Env variables hamesha text hote hain, port number chahiye
<code>HOSTNAME</code>	Kubernetes mein har pod ka HOSTNAME = pod ka naam. Isse pata chalta hai kis pod ne jawab diya
<code>settings = Settings()</code>	Ek object banao, aur baaki files from config import settings karti hain

Kahan se kya value aati hai:

Setting	Laptop pe	Kubernetes mein
<code>APP_VERSION</code>	<code>dev</code> (default)	1.0.0 (Dockerfile ka ENV , jo build-arg se aaya)
<code>APP_ENV</code>	<code>local</code> (default)	minikube (ConfigMap se)
<code>HOSTNAME</code>	<code>localhost</code>	Pod ka naam (Kubernetes set karta hai)

app/app.py

Metrics define karna

```
REQUESTS = Counter("flask_http_requests_total", "Total HTTP requests",
                   ["method", "endpoint", "status"])
LATENCY = Histogram("flask_http_request_duration_seconds", "HTTP request latency in seconds",
                   ,
                   ["endpoint"])
```

- **Counter** har request pe +1 hota hai, aur method/endpoint/status ke labels ke saath.
- **Histogram** response time ko buckets mein daalta hai (0.005s, 0.01s ... 10s), taaki p95 nikal sakein.
- Naam ke end mein **_total** aur **_seconds** Prometheus ka **naming convention** hai.

Har request ko measure karna

```
@app.before_request
def _start_timer():
    g.start = time.perf_counter()          # request shuru hone ka time

@app.after_request
def _record_metrics(response):
    endpoint = request.url_rule.rule if request.url_rule else "unmatched"
    if endpoint != "/metrics":
        REQUESTS.labels(request.method, endpoint, response.status_code).inc()
        LATENCY.labels(endpoint).observe(time.perf_counter() - g.start)
    return response
```

Cheez	Matlab
<code>before_request / after_request</code>	Har request se pehle / baad mein yeh function chalo (hooks)
<code>g</code>	Flask ka per-request storage
<code>request.url_rule.rule</code>	Route pattern (<code>/work</code>), poora URL nahi. Yeh cardinality se bachata hai
<code>"unmatched"</code>	404 wali requests ke liye ek hi label, taaki random URLs se lakhon series na banein
<code>!= "/metrics"</code>	Prometheus ke apne scrape ko count nahi karte, warna data mein noise aata

Routes

```
@app.get("/health")
def health():
    if _state["healthy"]:
        return jsonify(status="ok", pod=settings.hostname)
    return jsonify(status="unhealthy", pod=settings.hostname), 503
```

Normal mein 200 deta hai. `/fail` call hone ke baad `_state["healthy"] = False` ho jaata hai aur 503 aata hai. Yeh probes ka demo dikhane ke liye hai.

```
@app.get("/work")
def work():
    ms = min(request.args.get("ms", default=100, type=int), MAX_WORK_MS)
    deadline = time.perf_counter() + ms / 1000
    while time.perf_counter() < deadline:
        count += 1
```

Diye gaye milliseconds tak loop chala ke **CPU jalata hai**, taaki Grafana graph hile. `MAX_WORK_MS = 1000` ek safety cap hai, taaki koi `?ms=99999999` bhej ke server na atka de.

****_state dictionary kyun, simple variable kyun nahi?*** Dictionary ko function ke andar **global** likhe bina badal sakte hain. Yeh state **har process ka alag** hai, isliye ek pod pe `/fail` karne se sirf woh pod unhealthy hota hai.

app/gunicorn.conf.py

```
from config import settings
bind = f"0.0.0.0:{settings.port}" # port config.py se (DRY)
workers = 1 # scale = pods badhao, workers nahi
threads = 4 # ek process mein 4 requests ek saath
accesslog = "-" # "-" = stdout, taaki kubectl logs mein dikhe
```

Logs stdout pe kyun? Containers mein logs file mein nahi likhte. Stdout pe likho, aur Kubernetes/Docker unhe collect karta hai. `kubectl logs` yahi dikhata hai.

Ek worker kyun? (1) Kubernetes mein scaling pods se hoti hai. (2) `prometheus_client` ke counters ek process mein rehte hain. Kai workers hote to har scrape alag worker ka data deta aur graph galat aata.

Dockerfile

```
FROM python:3.12-slim
```

Official Python image, "slim" variant (Debian, bina extra tools). Size chhota hai, aur attack surface bhi kam.

```
ARG APP_VERSION=dev
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 APP_VERSION=${APP_VERSION}
```

- **ARG** sirf **build ke time** ka variable hai. `deploy.sh` isme `--build-arg APP_VERSION=1.0.0` bhejta hai.
- **ENV** container ke **run time** ka environment variable hai. ARG ki value ENV mein daal di, taaki app padh sake.
- **PYTHONDONTWRITEBYTECODE=1**: `.pyc` files mat banao (container mein bekaar hain).
- **PYTHONUNBUFFERED=1**: print turant dikhe, buffer mein na ruke. Iske bina `kubectl logs` mein logs der se aate.

```
WORKDIR /app
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ .
```

- **WORKDIR**: aage ke saare commands `/app` folder mein challenge.
- **Pehle requirements, phir code**. Yeh **layer caching** ki trick hai (Part 3).
- `--no-cache-dir`: pip ka download cache image mein na rahe, jisse image chhoti rehti hai.

```
RUN useradd --uid 10001 --no-create-home appuser
USER 10001
```

Root ki jagah normal user. Agar app mein security hole ho, to attacker ko root nahi milega. Kubernetes mein `runAsNonRoot: true` isko enforce karta hai: root se chalne ki koshish ki to pod start hi nahi hoga.

```
EXPOSE 5000
CMD ["gunicorn", "--config", "gunicorn.conf.py", "app:app"]
```

- **EXPOSE** sirf **documentation** hai, koi port nahi kholta.
- **CMD** container start hone pe yahi chalega. `app:app` ka matlab hai `app.py` file ka `app` object.
- ****JSON format ["..."] kyun?**** Isse gunicorn seedha PID 1 banta hai aur Kubernetes ke SIGTERM ko sahi se receive karta hai (graceful shutdown).

.dockerignore

Build ke time Docker poora folder (**build context**) leta hai. `.dockerignore` batata hai kya **mat** bhejo: `.git`, `.venv`, tests, docs. Isse build tez hota hai, image chhoti banti hai, aur secrets galti se image mein nahi jaate.

.gitignore aur .gitattributes

- **.gitignore**: `.venv/`, `__pycache__` aur cache Git mein na jaayein.
- **.gitattributes**: `* text=auto eol=lf`. Windows pe bhi files LF line endings ke saath save hon, taaki shell scripts Linux mein chalein.

k8s/configmap.yaml

```
kind: ConfigMap
metadata:
  name: flask-app-config
data:
  APP_ENV: minikube
```

Deployment mein `envFrom: configMapRef: flask-app-config` likha hai. Iska matlab hai ConfigMap ki **har key** container mein environment variable ban jaati hai.

k8s/deployment.yaml (sabse important file)

```
spec:
  replicas: 3
  selector:
    matchLabels:
      app: flask-app
```

"3 pods chahiye, aur mere pods woh hain jinpe **app: flask-app** label hai."

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 0
    maxSurge: 1
```

Update ke time kabhi bhi 3 se kam ready pods nahi, aur max 1 extra pod. Isse **zero downtime** milta hai.

```
template:
  metadata:
    labels:
      app: flask-app          # selector se match hona ZAROORI
    annotations:
      prometheus.io/scrape: "true"
```

Template = har pod ka blueprint. Label selector se match karna chahiye, warna Kubernetes error dega. Annotation Prometheus ko batata hai "mujhe scrape karo".

Label vs annotation: label pe selection hota hai (selectors). Annotation sirf extra jaankari hai, tools ke liye.

```
securityContext:
  runAsNonRoot: true
```

Container root se chalne ki koshish kare to Kubernetes rok dega.

```
containers:
- name: flask-app
  image: flask-app:1.0.0
  imagePullPolicy: IfNotPresent
```

IfNotPresent ka matlab hai image pehle se node pe hai to download mat karo. Hamari image Docker Hub pe hai hi nahi, isliye yeh zaroori hai. (**Always** hota to har baar download ki koshish karta aur fail hota.)

```
ports:
- name: http
  containerPort: 5000
```

Port ko **naam** diya: **http**. Aage sab jagah naam use hota hai, number nahi (DRY).

```
envFrom:
- configMapRef:
    name: flask-app-config
```

ConfigMap ki saari keys environment variables ban jaati hain.

```
resources:
  requests: {cpu: 100m, memory: 64Mi}
  limits:   {cpu: 250m, memory: 128Mi}
```

Flask idle mein ~30-40Mi memory leta hai, isliye 64Mi request aur 128Mi limit mein safe margin hai.

```

readinessProbe:
  httpGet: &health-check
    path: /health
    port: http
  initialDelaySeconds: 3
  periodSeconds: 5
  failureThreshold: 3
livenessProbe:
  httpGet: *health-check
    initialDelaySeconds: 10
    periodSeconds: 10
    failureThreshold: 3

```

- **&health-check** ek **YAML anchor** hai: "is block ko yeh naam do".
- ***health-check** ek **alias** hai: "wahi block yahan paste karo". Yeh DRY hai, YAML ka feature.
- **Readiness**: 3 sec baad shuru, har 5 sec, 3 fail, yaani ~15 sec mein traffic se hatega.
- **Liveness**: 10 sec baad shuru, har 10 sec, 3 fail, yaani ~30 sec mein restart.
- **Liveness slow kyun?** Restart bada action hai. Thodi der ki problem pe restart nahi chahiye.

k8s/service.yaml

```

spec:
  type: NodePort
  selector:
    app: flask-app
  ports:
    - name: http
      port: 80
      targetPort: http
      nodePort: 30080

```

app: flask-app label wale **ready** pods ko traffic bhejo. Cluster ke andar port 80, container pe **http** (5000), aur node pe 30080.

monitoring/00-namespace.yaml

monitoring namespace banata hai. ****00-** kyun? **kubectl apply -f monitoring/** files ko **alphabetical order** mein lagata hai, aur namespace pehle banna chahiye, warna Prometheus wali file fail hogi.

monitoring/10-prometheus.yaml

Ek file mein 6 objects hain, **---** se alag:

- **1. ServiceAccount prometheus:**** Prometheus pod ki identity.
- **2. ClusterRole prometheus:****

```

rules:
  - apiGroups: [""]
    resources: [nodes, nodes/proxy, nodes/metrics, services, endpoints, pods]
    verbs: [get, list, watch]

```

Sirf **padhne** ki permission. **nodes/proxy** cAdvisor tak pahunchne ke liye zaroori hai.

3. ClusterRoleBinding: ServiceAccount ko ClusterRole se jodta hai.

- **4. ConfigMap prometheus-config****, jisme **prometheus.yml** hai, yaani **kya scrape karna hai**:

```
global:
  scrape_interval: 15s
```

Har 15 second mein sab targets se data lo.

****Job 1: prometheus.**** Khud ko scrape karo (**localhost:9090**). Isse test hota hai ki scraping chal rahi hai.

****Job 2: kubernetes-cadvisor.**** Containers ka CPU/memory:

```
scheme: https
tls_config:
  ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
authorization:
  credentials_file: /var/run/secrets/kubernetes.io/serviceaccount/token
kubernetes_sd_configs:
  - role: node
relabel_configs:
  - target_label: __address__
    replacement: kubernetes.default.svc:443
  - source_labels: [__meta_kubernetes_node_name]
    target_label: __metrics_path__
    replacement: /api/v1/nodes/${1}/proxy/metrics/cadvisor
```

Line	Matlab
ca_file, credentials_file	Kubernetes har pod mein ServiceAccount ka token aur certificate is path pe daal deta hai. Prometheus inse API server ko apni pehchaan batata hai
role: node	API se saare nodes ki list lo
__address__ = kubernetes.default.svc:443	Seedha node pe nahi, API server pe jao
__metrics_path__	API server ka proxy path, jo node ke cAdvisor tak le jaata hai. \$1 = node ka naam

API server proxy kyun? Isse kubelet ke certificates ki jhanjhat nahi hoti, aur yeh har cluster pe same chalta hai.

****Job 3: app-pods.**** Hamare app ke **/metrics**:

```
kubernetes_sd_configs:
  - role: pod
relabel_configs:
  - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
    action: keep
    regex: "true"
  - source_labels: [__meta_kubernetes_pod_container_port_name]
    action: keep
    regex: http
  - source_labels: [__meta_kubernetes_namespace]
    target_label: namespace
  - source_labels: [__meta_kubernetes_pod_name]
    target_label: pod
```

Rule	Matlab
role: pod	Cluster ke saare pods lo (har pod ka har port ek candidate target)
keep annotation "true"	Sirf woh pods jinpe prometheus.io/scrape: "true" hai
keep port name http	Sirf http naam wala port. Port number yahan dobara nahi likha (DRY)
namespace, pod labels	Discovery ki info ko normal labels banao, taaki graph mein pod ka naam dikhe

__meta_... labels discovery se aate hain aur scrape ke baad gayab ho jaate hain. Isliye kaam ke labels ko copy kar lete hain.

****5. Deployment `prometheus`:**

- `serviceAccountName: prometheus`: upar wali identity use karo
- `image: prom/prometheus:v2.53.0`: fixed version
- `args`: config file kahan hai, data kahan store karna hai, 2 din ka data rakho
- Probes: `/-/ready` aur `/-/healthy` (Prometheus ke built-in endpoints)
- Volumes: config ConfigMap se `/etc/prometheus` pe, aur data `emptyDir` mein

****6. Service `prometheus`:** NodePort 30090. Grafana `http://prometheus:9090` se isi ko call karta hai.

monitoring/20-grafana.yaml

****ConfigMap `grafana-provisioning`** mein do files hain:

`datasources.yaml`:

```
datasources:
- name: Prometheus
  uid: prometheus
  type: prometheus
  url: http://prometheus:9090
  isDefault: true
```

Grafana start hote hi Prometheus data source ban jaata hai. URL mein **Service ka naam** hai, IP nahi (Kubernetes DNS). `uid` fixed rakha hai taaki dashboard JSON ise refer kar sake.

`dashboards.yaml`: `"/var/lib/grafana/dashboards` folder mein jo JSON files hain, unhe dashboards bana do."

****Deployment `grafana`:**

- `grafana/grafana:11.1.0`
- `subPath` mounts: ConfigMap ki ek-ek file ko sahi folder mein rakhta hai (datasources alag folder mein, dashboards provider alag mein)
- `grafana-dashboards` ConfigMap ko dashboards folder pe mount kiya. Yeh ConfigMap `deploy.sh` JSON file se banata hai

****Service `grafana`:** NodePort 30030.

monitoring/dashboards/flask-app.json

8 panels hain:

Panel	Type	Query ka matlab
Healthy pods being scraped	Stat	<code>sum(up{job="app-pods"})</code> : kitne pods ka <code>/metrics</code> mil raha hai. 3 = green, 2 = orange, 0-1 = red
Total requests / second	Stat	Saare pods ka total request rate
Total CPU used	Stat	Teeno pods ka total CPU (cores)
Total memory used	Stat	Teeno pods ki total memory
CPU usage per pod	Graph	Har pod ki alag line
Memory usage per pod	Graph	Har pod ki alag line
Requests per second by pod	Graph	Load balancing dikhta hai
95th percentile response time	Graph	Har endpoint ki p95 latency

"refresh": "10s" ka matlab hai har 10 sec mein update. "time": now-30m ka matlab hai pichle 30 minute dikhao.

scripts/deploy.sh

```
set -e || pipefail
```

Safety line. -e = koi command fail ho to ruk jao. -u = undefined variable error hai. -o pipefail = pipe mein koi bhi command fail ho to fail maano. Iske bina script galti ke baad bhi chalti rehti.

```
IMAGE=$(awk ' $1 == "image:" {print $2; exit}' k8s/deployment.yaml)
VERSION=${IMAGE##*:}
```

DRY ka example. Image ka naam deployment.yaml se **padha**, dobara likha nahi. \${IMAGE##*:} ka matlab hai : ke baad wala hissa, yaani 1.0.0.

```
minikube status >/dev/null 2>&1 || minikube start --driver=docker --cpus=2 --memory=4096
```

|| = pehli command fail ho tabhi doosri chalao. Yaani Minikube chal raha hai to dobara start mat karo.
>/dev/null 2>&1 = output chhupao.

```
docker build --build-arg APP_VERSION="$VERSION" -t "$IMAGE" .
minikube image load "$IMAGE"
kubectl apply -f k8s/
kubectl apply -f monitoring/
kubectl create configmap grafana-dashboards -n monitoring \
  --from-file=monitoring/dashboards/ --dry-run=client -o yaml | kubectl apply -f -
```

Last command ki trick: create --dry-run=client -o yaml **YAML banata hai par lagata nahi**, aur | **kubectl apply -f -** usse lagata hai. Seedha create doosri baar "already exists" error deta, apply update kar deta hai. Isse script **idempotent** banti hai: kitni baar bhi chalao, result same.

```
kubectl rollout status deployment/flask-app --timeout=120s
```

Tab tak ruko jab tak saare pods ready na ho jaayein.

scripts/load.sh

```
kubectl run load-generator --rm -it --restart=Never --image=busybox:1.36 -- /bin/sh -c "..."
```

- Cluster ke **andar** ek temporary busybox pod (chhota Linux toolbox)
- rm = khatam hone pe pod delete. --restart=Never = Deployment nahi, sirf ek pod
- Andar **wget** loop <http://flask-app-service/work?ms=50> ko call karta hai, **Service ke naam se** (DNS)
- Parallel clients: **for i in \$(seq N); do ... & done; wait.** & = background mein chalao
- winpty**: Git Bash pe interactive commands ke liye zaroori

Andar se load kyun? Bahar se tunnel ke through bhejte to tunnel bottleneck banta, aur load balancing bhi saaf nahi dikhta.

scripts/teardown.sh

`kubectl delete -f k8s/` aur `kubectl delete -f monitoring/` chalata hai. `--ignore-not-found` lagaya hai taaki pehle se delete ho to error na aaye. Namespace delete karne se uske andar sab kuch (grafana-dashboards bhi) delete ho jaata hai.

scripts/build_pdf.py

Yeh PDF isi script se bani hai. Markdown files (`docs/`) padhta hai, unhe PDF format mein badalta hai, aur end mein asli source files jodta hai. **PDF ka apna koi content nahi hai**, isliye docs badlo aur script chalao to PDF update ho jaati hai (DRY).

```
python scripts/build_pdf.py          # dono PDF (English + Hinglish)
python scripts/build_pdf.py hinglish # sirf yeh wali
```

tests/

`**test_app.py**` (5 tests):

Test	Kya check karta hai
homepage	/ 200 deta hai aur app ka naam dikhati hai
health ok	/health 200 + "status": "ok"
fail -> 503	/fail ke baad /health 503
metrics	Request ke baad counter /metrics mein dikhti hai
work capped	?ms=999999 pe bhi max 1000ms

`**test_manifests.py` (5 tests, bina cluster ke**):

Test	Kya pakadta hai
Selector = labels	Deployment apne pods pehchan payega ya nahi
Service sahi pods + port	Service ka selector kisi Deployment se match karta hai, aur <code>targetPort</code> ka naam container mein hai
Probes same	Readiness aur liveness same health check use karte hain, resources set hain
Prometheus config valid	<code>prometheus.yml</code> sahi YAML hai aur <code>app-pods</code> job hai
Dashboard datasource	Dashboard ka uid provisioning ke uid se match karta hai

`**pytest.ini:**` `pythonpath = app` taaki tests `import app` kar sakein. **`**requirements-dev.txt:**`** testing aur PDF ke tools. Runtime libraries `-r app/requirements.txt` se aati hain, dobara nahi likhi (DRY).

Aise tests kyun? Galti cluster pe deploy karke pakadne mein 5 minute lagte hain, test se 2 second. Yeh **"shift left"** hai: galti jitni jaldi pakdo, utna sasta.

Part 6 - Project Banate Waqt Kya-Kya Kiya Gaya

Yeh part project ki **history** hai: kis order mein kya bana aur kyun. Interview mein "aapne project kaise banaya?" ka jawab yahi hai.

Step 1 - Laptop check kiya

Sabse pehle dekha ki laptop pe kya installed hai:

Tool	Mila?
Python 3.14	Haan
Git 2.54	Haan
Minikube v1.38	Haan
kubectl v1.36	Haan
Docker	Nahi
WSL2	Nahi
RAM / CPU	32 GB / 12 threads (kaafi hai)

Seekh: kaam shuru karne se pehle environment check karo. Andaze pe mat chalo.

Git ki global identity bhi check ki, aur repo mein author **Radhika Agarwal** <theradhika2410@gmail.com> set kiya.

Step 2 - Project folder aur Git repo

- Folder banaya: `C:\Users\sunil\projects\k8s-flask-monitoring`
- `git init -b main`: repo banaya, branch ka naam `main`
- Folders banaye: `app`, `tests`, `k8s`, `monitoring/dashboards`, `docs/screenshots`, `scripts`
- `.venv` banaya aur libraries install ki: Flask, prometheus-client, gunicorn, pytest, pyyaml, reportlab, svglib

Step 3 - Phase 1: architecture diagram (commit 1)

`docs/architecture.svg` banaya. SVG ek text-based image format hai, jo GitHub pe seedha dikhta hai aur Git mein diff bhi hota hai. Saath mein `.gitignore` bhi banaya.

Step 4 - Phase 2: Flask app (commit 2)

`config.py`, `app.py`, `gunicorn.conf.py`, `requirements.txt` aur tests likhe.

Check kaise kiya:

1. `python -m pytest` chalaya, aur 5 tests pass hue
2. App ko local chalaya (`PORT=5055 python app.py`) aur `curl` se `/health`, `/work`, `/metrics` aur `/` check kiye. Sab sahi jawab de rahe the

Seekh: pehle chhote level pe test karo (local), phir bade pe (Docker, Kubernetes).

Step 5 - Phase 3: Docker (commit 3)

Dockerfile, .dockerignore aur .gitattributes banaye.

Ek problem aayi: commit karte waqt Git ne warning di: `"LF will be replaced by CRLF"`. Yeh Windows ki line endings ki problem hai. Agar shell scripts CRLF ke saath jaate to Linux container mein toot jaate. **Solution:** `.gitattributes` mein `eol=lf` likha aur `git add --renormalize .` chalaya.

Step 6 - Phases 5-6: Kubernetes YAML (commit 4)

`configmap.yaml`, `deployment.yaml` (replicas, probes, limits, rolling update) aur `service.yaml` banaye. Saath mein `test_manifests.py` likha, jo cluster ke bina YAML ki galtiyen pakadta hai.

Step 7 - Phases 7-8: monitoring (commit 5)

`00-namespace.yaml`, `10-prometheus.yaml`, `20-grafana.yaml` aur `dashboards/flask-app.json` banaye.

Design decisions:

Decision	Kyun
Helm nahi, plain YAML	Har line samajh aaye
Files pe number (00-, 10-, 20-)	Apply ka order sahi rahe
cAdvisor API server proxy se	Har cluster pe chale, certificate ki jhanjhat nahi
Pods ko port naam se scrape kiya	Port number do jagah na likhna pade
Dashboard alag JSON file mein	Ek hi source; Grafana UI mein import bhi ho sakta hai
Dashboard descriptions mein limits ke numbers nahi likhe	Woh deployment.yaml mein hain. Do jagah hote to ek din alag ho jaate

Step 8 - Scripts (commit 6)

`deploy.sh`, `load.sh` aur `teardown.sh` likhe. `bash -n` se syntax check kiya. `git update-index --chmod=+x` se Git mein executable mark kiya, taaki Linux/Mac pe seedha `./scripts/deploy.sh` chale.

Ek sudhaar: `load.sh` pehle sirf ek client chalata tha. Socha to samajh aaya ki ek client se CPU limit tak kabhi nahi pahunchenge (ek baar mein ek hi request, ek hi pod pe). Isliye **parallel clients** ka option joda: `./scripts/load.sh 500 6`.

Ek aur sudhaar: OOM test mein pehle sirf memory limit 20Mi kam ki thi. Lekin request (64Mi) limit se zyada nahi ho sakti, aur Kubernetes woh change reject kar deta. Isliye command mein request aur limit dono 16Mi kiye.

Step 9 - Phases 9-10: docs aur PDF (commit 7)

- `README.md`: GitHub pe sabse pehle yahi dikhta hai
- `docs/guide.md`: 10 phases, har command ki explanation (English)
- `docs/troubleshooting.md` aur `docs/interview-prep.md`
- `scripts/build_pdf.py` se English PDF banaya

PDF check kiya: pages ko image mein render karke dekha. Do problems mili:

1. Diagram ke kuch labels lines ke upar aa rahe the, to SVG mein positions badli

2. Kuch headings page ke end mein akeli reh jaati thi aur content agle page pe chala jaata tha. Headings ko `keepWithNext` diya

Step 10 - Author theek kiya

Pehle commits galti se doosre naam se ban gaye the. Push se **pehle** saare 7 commits ka author badla:

```
git config user.name "Radhika Agarwal"
git config user.email "theradhika2410@gmail.com"
git rebase --root --exec "git commit --amend --no-edit --reset-author"
```

`rebase --root --exec` ka matlab hai pehle commit se shuru karke har commit pe yeh command chalao. `--reset-author` = author ko current config se badlo.

Zaroori: history rewrite **sirf push se pehle** karna safe hai. Push ke baad karoge to doosron ki copy se mismatch hoga aur force push karna padega.

Step 11 - GitHub pe push

```
git remote add origin https://github.com/radhikaa06/k8s-flask-monitoring.git
git ls-remote origin          # check: repo khaali hai na?
git push -u origin main
```

7 commits GitHub pe pahunch gaye.

Step 12 - Hinglish PDF (yeh document)

`docs/hinglish/` mein 10 Markdown files likhi, aur `build_pdf.py` mein "edition" ka option joda taaki same script dono PDF banaye (DRY).

Abhi kya baaki hai

Kaam	Status	Kyun
Code, YAML, scripts, docs	Ho gaya	-
Tests (10)	Pass	-
GitHub push	Ho gaya	-
Minikube pe asli deploy	Baaki	Laptop pe Docker aur WSL2 nahi hain
Grafana screenshots	Baaki	Deploy ke baad hi milenge
GitHub description/topics	Baaki	Website pe 2 minute ka kaam

Imaandari se: abhi tak code ko cluster pe chala ke verify **nahi** kiya gaya hai. Tests sirf app aur YAML ki consistency check karte hain. Asli verification Part 7 follow karke hoga. Usme koi error aaye to Part 8 ka troubleshooting use karo.

Part 7 - Project Chalana: Step by Step

Terminals:

Naam	Kaise kholein	Kis liye
PowerShell (Admin)	Start, "PowerShell" type karo, right-click, "Run as administrator"	Sirf WSL install
PowerShell	Normal	Docker install
Git Bash	Project folder mein right-click, "Open Git Bash here"	Baaki sab kuch

Step 1 - WSL2 install

```
wsl --install
```

- **Kya:** Windows Subsystem for Linux 2 aur Ubuntu install karta hai.
- **Kyun:** Containers Linux kernel ka feature hain. Docker Desktop WSL2 ke andar chalta hai.
- **Kahan:** PowerShell **as Administrator**.
- **Expected:** The requested operation is successful. Changes will not be effective until the system is rebooted. Phir **restart** karo. Ubuntu window khulegi aur username/password maangegi, jo kuch bhi rakh lo.
- **Error aaye to:**
 - **0x80370102:** BIOS mein virtualization off hai. BIOS mein **Intel VT-x** ya **AMD-V/SVM** enable karo. Task Manager, Performance, CPU mein "Virtualization: Enabled" dikhna chahiye.
 - "requires elevation": PowerShell Admin se nahi khola.
 - 0% pe atka: **wsl --install --web-download** chalao.

Check karo:

```
wsl --status
```

Expected: Default Version: 2.

Step 2 - Docker Desktop install

```
winget install -e --id Docker.DockerDesktop
```

- **Kya:** Windows ke package manager se Docker Desktop install karta hai.
- **Kahan:** Normal PowerShell.
- **Iske baad:** Docker Desktop kholo, terms accept karo, aur neeche-left mein **"Engine running"** (hara) aane tak ruko.

Check karo (**naye** Git Bash mein):

```
docker run hello-world
```

- **Expected:** Hello from Docker!
- **Error:** error during connect matlab Docker Desktop chalu nahi hai. **docker: command not found** matlab terminal purana hai, naya kholo.

Step 3 - Project laptop pe hai na?

Project pehle se `C:\Users\sunil\projects\k8s-flask-monitoring` mein hai. Kisi aur laptop pe ho to:

```
git clone https://github.com/radhikaa06/k8s-flask-monitoring.git
cd k8s-flask-monitoring
```

Step 4 - Tests chalao (optional, bina cluster ke)

```
python -m venv .venv
source .venv/Scripts/activate
pip install -r requirements-dev.txt
python -m pytest
```

- **Expected:** 10 passed.
- **Error:** `ModuleNotFoundError` matlab `venv activate` nahi hai. Prompt ke aage `(.venv)` dikhna chahiye.

Step 5 - Ek command mein sab deploy

```
./scripts/deploy.sh
```

- **Kya karta hai (andar):**
 1. Minikube start (2 CPU, 4 GB RAM), agar pehle se nahi chal raha
 2. metrics-server addon on
 3. `docker build`, jo image `flask-app:1.0.0` banata hai
 4. `minikube image load`, jo image Minikube mein bhejta hai
 5. `kubectl apply -f k8s/`, jo app deploy karta hai
 6. `kubectl apply -f monitoring/`, jo Prometheus aur Grafana deploy karta hai
 7. Dashboard ConfigMap banata hai
 8. Sab ready hone ka wait karta hai
- **Kahan:** Git Bash, project folder. Docker Desktop chalu hona chahiye.
- **Time:** pehli baar 5-10 minute (Kubernetes aur images download hote hain).
- **Expected:** end mein `Done`. aur teen `minikube service` commands dikhenge.
- **Error aaye to:**

Error	Matlab	Solution
<code>PROVIDER_DOCKER_NOT_RUNNING</code>	Docker band hai	Docker Desktop kholo
Memory ka error	Docker ke paas kam RAM	Docker Settings, Resources mein RAM badhao, ya <code>deploy.sh</code> mein <code>--memory=3072</code> karo
<code>Permission denied</code>	Script executable nahi	<code>bash scripts/deploy.sh</code> chalao
<code>\$'\r': command not found</code>	Windows line endings	<code>git add --renormalize .</code>
<code>rollout status</code> timeout	Pods ready nahi hue	<code>kubectl get pods -A</code> dekho, phir Part 8

Step 6 - Sab chal raha hai? Check karo

```
kubectl get pods -A
```

- **Kya:** saare namespaces (-A) ke pods dikhao.
- **Expected:**

NAMESPACE	NAME	READY	STATUS	RESTARTS
default	flask-app-6b7c9d8f4-abcde	1/1	Running	0
default	flask-app-6b7c9d8f4-fghij	1/1	Running	0
default	flask-app-6b7c9d8f4-klmno	1/1	Running	0
monitoring	grafana-5f6d7c8b9-pqrst	1/1	Running	0
monitoring	prometheus-7a8b9c0d1-uvwxy	1/1	Running	0
kube-system	...	...	Running	...

READY 1/1 ka matlab hai pod mein 1 container hai aur woh ready hai.

```
kubectl get deployment,service,endpoints
```

- **Expected:** flask-app 3/3, flask-app-service NodePort 80:30080/TCP, aur endpoints mein 3 IPs.

Step 7 - App kholo

```
minikube service flask-app-service --url
```

- **Kya:** tunnel bana ke URL deta hai.
- **Kahan:** Git Bash. **Yeh terminal khula rakho**, band kiya to tunnel band.
- **Expected:** <http://127.0.0.1:54321> jaisa URL. Browser mein kholo, aur "Hello from flask-app" ke saath pod ka naam dikhega.

Load balancing dekho (naye terminal mein, URL apna daalo):

```
for i in 1 2 3 4 5 6; do curl -s http://127.0.0.1:54321/health; echo; done
```

Expected: pod ki value alag-alag pods ke naam dikhayegi. (Browser ek connection reuse karta hai, isliye wahan aksar same pod dikhta hai.)

Step 8 - Prometheus kholo

```
minikube service prometheus -n monitoring --url
```

URL kholo, phir **Status > Targets**.

Expected:

- prometheus (1/1 up)
- kubernetes-cadvisor (1/1 up)
- app-pods (3/3 up)

Graph tab mein yeh queries try karo: `up, flask_http_requests_total, sum by (pod) (rate(flask_http_requests_total[1m]))`.

Step 9 - Grafana kholo

```
minikube service grafana -n monitoring --url
```

- Login: **admin** / **admin**. Grafana naya password maangega, kuch bhi rakh lo (yaad rakhna).
- **Dashboards > Flask App on Kubernetes** kholo.
- **Expected:** "Healthy pods" = 3 (green). Baaki graphs idle mein lagbhag flat honge.

Step 10 - Load do aur graphs dekho

```
./scripts/load.sh 50
```

- 30-60 second mein graphs hilne lagenge: requests/sec badhega, CPU badhega, aur teeno pods ki lines dikhengi.
- **Ctrl+C** se band karo.

Step 11 - Screenshots lo

`docs/screenshots/` mein **inhi naamon** se save karo (README inhi naamon ke links deta hai):

File	Kya capture karna hai
<code>grafana-dashboard.png</code>	Poora dashboard, load ke time
<code>prometheus-targets.png</code>	Status > Targets, sab up
<code>Pods-running.png</code>	Terminal mein <code>kubectl get pods -A</code>
<code>self-healing.png</code>	Pod delete test (Part 8, Test 1)

Windows pe screenshot: **Win + Shift + S**.

Phir push karo:

```
git add docs/screenshots
git commit -m "Add monitoring screenshots"
git push
```

Step 12 - Kaam khatam? Band karo

```
./scripts/teardown.sh    # app aur monitoring hatao
minikube stop            # cluster pause (data rehta hai)
```

Agli baar **minikube start**, phir `./scripts/deploy.sh`. Cluster poora hatana ho to **minikube delete**.

Code badla to dobara deploy kaise karein?

1. `k8s/deployment.yaml` mein image tag badlo: **flask-app:1.0.1**
2. `./scripts/deploy.sh` chalao

Tag kyun badalna? Same tag pe Minikube ke paas purani image hai aur **IfNotPresent** ki wajah se nayi nahi lega. Naya tag = nayi image = rolling update. Version **sirf ek jagah** (`deployment.yaml`) badalna hai, script baaki khud padh lega.

Part 8 - Todna aur Theek Karna: Experiments aur Troubleshooting

Asli seekh yahin hai. Project chalana aasaan hai. Interview mein farq is baat se padta hai ki aapne cheezein **todi** hain, **dekha** kya hota hai, aur **theek** kiya hai.

Har experiment ke time Grafana dashboard aur terminal **saath-saath** khula rakho.

Test 1 - Self-healing: pod delete karo

Terminal 1 (watch mode):

```
kubectl get pods -l app=flask-app -w
```

-l app=flask-app = sirf is label wale pods. **-w** = watch, yaani badlav live dikhao.

Terminal 2:

```
kubectl delete pod <koi-ek-flask-app-pod-ka-naam>
```

Kya dikhega:

```
flask-app-xxx-abcde    1/1    Terminating
flask-app-xxx-zzzzz    0/1    Pending
flask-app-xxx-zzzzz    0/1    ContainerCreating
flask-app-xxx-zzzzz    0/1    Running      <- container chal gaya, readiness abhi pass nahi
flask-app-xxx-zzzzz    1/1    Running      <- ready, ab traffic milega
```

Kyun hua? ReplicaSet controller ne dekha: chahiye 3, hain 2, to naya banao. Naye pod ka **naam alag** hai. Pods **disposable** hain ("cattle, not pets").

Grafana mein: "Healthy pods" thodi der ke liye 2 (orange), phir 3 (green).

Aur aage: `kubectl delete pods -l app=flask-app` se teeno ek saath delete karo, aur teeno wapas aa jaayenge.

Test 2 - Liveness probe: app ko "beemar" karo

Terminal 1:

```
kubectl port-forward deployment/flask-app 8080:5000
```

port-forward laptop ke port 8080 ko **ek** pod ke 5000 se jodta hai. Service ko bypass karta hai, taaki pata ho kaunsa pod beemar kiya.

Terminal 2:

```
curl -X POST localhost:8080/fail
kubectl get pods -l app=flask-app -w
```

Kya dikhega:

- ~15 sec: woh pod **0/1** ho jaata hai. **Readiness fail** hua, to Service ne traffic bhejna band kiya. Restart nahi hua.
- ~30 sec: **RESTARTS** 0 se 1 hota hai. **Liveness fail** hua, to kubelet ne container restart kiya.

- Restart ke baad naya process healthy hai, to **1/1** wapas.

Saboot dekho:

```
kubectl describe pod <us-pod-ka-naam>
```

Neeche **Events** mein:

```
Warning Unhealthy Readiness probe failed: HTTP probe failed with statuscode: 503
Warning Unhealthy Liveness probe failed: HTTP probe failed with statuscode: 503
Normal Killing Container flask-app failed liveness probe, will be restarted
```

Seekh: Readiness **users ko bachata hai** (kharab pod pe traffic nahi jaata). Liveness pod ko **theek karta hai** (restart).

Note: port-forward restart ke baad toot sakta hai ("lost connection to pod"). Yeh normal hai, dobara chalao.

Test 3 - Kharab release: galat image

```
kubectl set image deployment/flask-app flask-app=flask-app:9.9.9
kubectl get pods -l app=flask-app
kubectl rollout status deployment/flask-app --timeout=30s
```

set image = Deployment ki image badlo (container **flask-app** ki image **flask-app:9.9.9** karo). Yeh version exist nahi karta.

Kya dikhega:

flask-app-NEW-xxxxx	0/1	ErrImagePull	<- naya pod, image nahi mili
flask-app-OLD-aaaaa	1/1	Running	<- purane 3 abhi bhi chal rahe!
flask-app-OLD-bbbbb	1/1	Running	
flask-app-OLD-cccc	1/1	Running	

rollout status timeout ho jaayega.

Site down kyun nahi hui? **maxUnavailable: 0**. Naya pod ready nahi hua, isliye Kubernetes ne purana ek bhi nahi hataya. **Rolling update ki safety yahi hai**.

Theek karo:

```
kubectl rollout undo deployment/flask-app
```

Expected: **deployment.apps/flask-app rolled back**. Kharab pod gayab.

History dekho: **kubectl rollout history deployment/flask-app**

Test 4 - Memory limit: OOMKilled

```
kubectl set resources deployment flask-app --requests=memory=16Mi --limits=memory=16Mi
kubectl get pods -l app=flask-app -w
```

Request bhi 16Mi isliye kiya kyunki **request kabhi limit se zyada nahi ho sakti**. Sirf limit kam karte to Kubernetes reject kar deta.

Kya dikhega: naye pods **OOMKilled**, phir **CrashLoopBackOff**.

```
kubectl describe pod <naya-pod>
```

```
Last State: Terminated
Reason: OOMKilled
Exit Code: 137
```

137 = 128 + 9 (SIGKILL). Linux kernel ne memory limit cross hone pe process maar diya.

CrashLoopBackOff kya hai? Container baar-baar crash ho raha hai, isliye Kubernetes har baar restart se pehle zyada der rukta hai: 10s, 20s, 40s... max 5 minute.

Purane pods phir se chal rahe hain (Test 3 jaisa).

Theek karo: `kubectl rollout undo deployment/flask-app` ya `kubectl apply -f k8s/` (Git wala version wapas).

Test 5 - CPU limit: throttling

```
./scripts/load.sh 500 6
```

6 parallel clients, aur har request 500ms CPU maangti hai. Itna CPU 3 pods x 0.25 core se nahi milega.

Grafana mein kya dikhega:

- "CPU usage per pod" ki har line **0.25 pe flat** ho jaayegi, upar nahi jaayegi
- "95th percentile response time" **badh** jaayega
- **Koi restart nahi**

Seekh: CPU limit cross karne pe process **slow** hota hai (throttle), **marta nahi**. Memory pe **marta hai**. Yeh farq interview mein zaroor poocha jaata hai.

Test 6 - Load mein scale karo

`load.sh` chalte waqt:

```
kubectl scale deployment flask-app --replicas=5
```

Grafana: "Requests per second by pod" mein 2 nayi lines aayengi, aur har pod ka CPU kam hoga (load baant gaya).

Wapas: `kubectl scale deployment flask-app --replicas=3`.

Dhyan do: `kubectl scale` sirf live cluster badalta hai. YAML mein abhi bhi 3 hai. Agla `kubectl apply 3` kar dega. **Git wali file = sach**. Is soch ko **GitOps** ki neev samjho.

Troubleshooting ka tareeka (5 steps)

Koi bhi problem ho, **isi order** mein chalo:

Step	Command	Kya pata chalega
1. Get	<code>kubectl get pods -A</code>	Kaunsa pod pareshan hai? STATUS? RESTARTS?
2. Describe	<code>kubectl describe pod <naam></code>	Events (sabse neeche): image, scheduling, probes, OOM
3. Logs	<code>kubectl logs <naam></code> (crash ke baad <code>--previous</code>)	App ne khud kya error print kiya
4. Events	<code>kubectl get events --sort-by=.lastTimestamp</code>	Cluster mein abhi kya-kya hua, time ke order mein

Step	Command	Kya pata chalega
5. Exec	<code>kubectl exec -it <naam> -- sh</code>	Container ke andar jaake dekho (files, env, network)

Network problem ho to 2 aur checks:

Check	Command	Sahi jawab
Service ke peeche pods hain?	<code>kubectl get endpoints flask-app-service</code>	3 IPs
Labels match karte hain?	<code>kubectl get pods --show-labels</code>	<code>app=flask-app</code>

****--previous kyun?*** Crash ke baad naya container chal raha hota hai, aur `kubectl logs` naye ke logs dikhata hai. Crash ka error **purane** container mein tha.

Error catalogue

STATUS / Problem	Matlab	Kaise confirm karein	Solution
<code>ErrImagePull / ImagePullBackOff</code>	Image nahi mili	describe: <code>not found / pull access denied</code>	<code>minikube image load flask-app:1.0.0</code> , tag check karo
<code>CrashLoopBackOff</code>	Container start hote hi band	<code>kubectl logs <pod> --previous</code>	Logs ka error theek karo
<code>OOMKilled (137)</code>	Memory limit cross	describe: <code>Reason: OOMKilled</code>	Limit badhao ya memory leak dhoondho
<code>Pending</code>	Kisi node pe jagah nahi (requests ke hisaab se)	describe: <code>Insufficient cpu/memory</code>	Requests kam karo, ya Minikube ko zyada CPU/RAM do
<code>CreateContainerConfigError</code>	ConfigMap/Secret nahi mila	describe: <code>configmap not found</code>	<code>kubectl apply -f k8s/configmap.yaml</code>
<code>ContainerCreating atka</code>	Volume mount nahi ho raha	describe: <code>FailedMount</code>	Missing ConfigMap banao (jaise <code>grafana-dashboards</code>)
<code>Running</code> lekin <code>0/1</code>	Readiness fail	describe: <code>Readiness probe failed</code>	<code>port-forward</code> se <code>/health</code> check karo, probe ka path/port dekho
RESTARTS badh rahe	Liveness fail ya crash	describe: <code>failed liveness probe</code>	<code>/health</code> time pe jawab de; slow start ho to <code>initialDelaySeconds</code> badhao
Browser mein nahi khul raha	Tunnel band ya endpoints khaali	<code>kubectl get endpoints</code>	<code>minikube service</code> wala terminal khula rakho; selector theek karo
<code>connection refused</code> (kubectl)	Cluster band / galat context	<code>kubectl config current-context</code>	<code>minikube start</code>
Rollout atka	Naye pods ready nahi	<code>kubectl rollout status</code>	<code>kubectl rollout undo</code> , phir naye pod ki jaanch
Prometheus target DOWN	Pod ready nahi / port galat	Status > Targets ka error	Port ka naam <code>http</code> ho, <code>/metrics</code> chal raha ho
cAdvisor <code>403 Forbidden</code>	RBAC permission nahi	Status > Targets	ClusterRole mein <code>nodes/proxy; 10-prometheus.yaml</code> dobara apply
Grafana <code>No data</code>	Query galat / time range / traffic nahi	Query Prometheus Graph mein chalao	<code>load.sh</code> chalao, time range "Last 15 minutes"
namespaces <code>"monitoring" not found</code>	Order galat	-	<code>kubectl apply -f monitoring/</code> dobara chalao

Kaam ki commands

kubectl get all -n monitoring	# namespace ki saari cheezein
kubectl logs -l app=flask-app --tail=20 --prefix	# teeno pods ke last 20 logs
kubectl top pods -A	# abhi ka CPU/memory
kubectl get pod <naam> -o yaml	# pod ki poori live config
kubectl explain deployment.spec.strategy	# kisi bhi field ki documentation
minikube dashboard	# Kubernetes ka web UI

Part 9 - Interview Preparation

Interview **English** mein hoga, isliye har sawaal ke do hisse hain:

- **Samjho:** Hinglish mein concept
- **Bolo:** interview mein bolne layak English answer

Ratna mat. Samajh ke **apne shabdon** mein bolo. Upar wale experiments khud kiye honge to answers apne aap natural lagenge.

60-second project pitch

Bolo:

"I built a small Flask web application and focused on the DevOps around it. I containerised it with a slim, non-root Docker image and deployed it to a local Kubernetes cluster using Minikube. The Deployment runs three replicas with readiness and liveness probes and CPU and memory limits, and a NodePort Service load-balances traffic across them. For monitoring, I set up Prometheus and Grafana using plain YAML. Prometheus discovers pods through the Kubernetes API and scrapes container CPU and memory from cAdvisor, plus request metrics from the app's /metrics endpoint. Grafana shows them on a dashboard that is provisioned from code. Then I tested failures: I deleted pods, made the health check fail, deployed a broken image and set the memory limit too low. I diagnosed each with kubectl describe, logs and events, and recovered using rollbacks."

Q1. Why is Kubernetes used?

Samjho: Ek container chalana easy hai, lekin bahut saare containers ko reliably chalana mushkil hai: restart, scaling, load balancing, bina downtime update. Kubernetes yeh sab automatic karta hai. Aap "kya chahiye" likhte ho, aur woh lagatar wahi banaye rakhta hai.

Bolo: "Kubernetes is a container orchestrator. Running one container with docker run is easy, but running many containers reliably is hard. With Kubernetes I declare the desired state, for example three replicas of this image with these resource limits, and its controllers continuously make the actual state match it. It restarts failed containers, replaces lost pods, schedules pods on nodes with free capacity, load-balances through Services, and does rolling updates and rollbacks without downtime."

Q2. Difference between Pod, Deployment and Service?

Samjho: Pod = ek waiter (temporary). Deployment = manager jo 3 waiter hamesha rakhta hai. Service = restaurant ka fixed phone number.

Bolo: "A Pod is the smallest deployable unit, one or more containers sharing an IP address. Pods are ephemeral: if one dies, it is replaced by a new pod with a new name and IP. A Deployment manages pods through a ReplicaSet. It keeps the desired number of replicas and handles rolling updates and rollbacks. A Service gives a stable IP and DNS name, and load-balances traffic to all ready pods that match its label selector. In my project the Deployment creates three pods labelled app equals flask-app, and the Service selects that label."

Q3. How do replicas work?

Samjho: **replicas: 3** = desired state. ReplicaSet controller ginti karta rehta hai aur kam ho to banata hai, zyada ho to hatata hai. Isko **reconciliation loop** kehte hain.

Bolo: "Replicas is the desired number of identical pods. The Deployment creates a ReplicaSet, and its controller runs a reconciliation loop: it counts the pods matching its selector and creates or deletes pods until the count matches. Replicas give high availability, because one pod can fail without an outage, and capacity, because the Service spreads requests across them. I could see this in Grafana in my requests-per-pod panel."

Q4. How does Kubernetes handle pod failure?

Samjho: Do level: (1) container crash ho ya liveness fail ho, to kubelet **wahi pod** mein container restart karta hai. (2) Pod hi delete ho jaaye ya node mar jaaye, to ReplicaSet **naya pod** banata hai. Saath mein readiness kharab pod ko traffic se hata deta hai.

Bolo: "At two levels. If a container crashes or fails its liveness probe, the kubelet restarts that container inside the same pod, and the restart count goes up. Repeated crashes lead to CrashLoopBackOff with increasing delays. If a pod is deleted or its node is lost, the ReplicaSet notices fewer pods than desired and creates a replacement pod. Meanwhile, the readiness probe removes unhealthy pods from the Service, so users are not sent to them. I tested this by deleting a pod and watching a new one become ready within seconds."

Q5. Liveness vs readiness probe?

Samjho: Readiness = "traffic de sakte hain?", fail hone pe Service se hatao. Liveness = "zinda hai?", fail hone pe restart. Liveness mein database check **kabhi mat** dalo.

Bolo: "A readiness probe answers: can this pod receive traffic right now? If it fails, the pod is removed from the Service endpoints but not restarted. That is useful during startup or temporary overload. A liveness probe answers: is this container broken beyond recovery? If it fails, the kubelet restarts the container, which is useful for deadlocks. In my project both call the /health endpoint, but liveness is slower and more tolerant, because a restart is the more disruptive action. A common mistake is checking a database in the liveness probe: if the database goes down, every pod restarts in a loop."

Q6. How does Prometheus collect metrics?

Samjho: Pull model, har 15 second. Targets Kubernetes API se discover hote hain (RBAC chahiye). Time series = naam + labels + value + time.

Bolo: "Prometheus uses a pull model. Every fifteen seconds it sends an HTTP request to each target's metrics endpoint and stores the values as time series, identified by a metric name and labels. It finds targets through Kubernetes service discovery by asking the API server for pods and nodes, which is why it needs RBAC permissions. Relabeling rules decide which targets to keep. I scrape cAdvisor in the kubelet for container CPU and memory, and my app's /metrics endpoint, where I use the prometheus_client library to expose a request counter and a latency histogram."

Q7. How does Grafana visualise data?

Samjho: Grafana data store nahi karta. Prometheus se PromQL query karke graph banata hai. Humne provisioning se data source aur dashboard files se banaye.

Bolo: "Grafana does not store metrics itself. It connects to Prometheus as a data source, sends PromQL queries for the selected time range, and renders the results as panels like time-series graphs and stat panels. In my project the data source and the dashboard are provisioned from files in ConfigMaps, so they are version-controlled in Git and come back automatically if Grafana restarts."

Q8. How do you troubleshoot a failed deployment?

Samjho: get, describe (events), logs (--previous), endpoints, theek karo ya rollout undo.

Bolo: "I follow the same order every time. First, kubectl get pods to see the status, such as ImagePullBackOff, CrashLoopBackOff, Pending, or zero of one ready. Second, kubectl describe pod, because the events at the bottom usually say exactly what failed. Third, kubectl logs with --previous if the container crashed. If the app runs but cannot be reached, I check the Service endpoints, because empty endpoints mean a label mismatch or no ready pods. To recover quickly I use kubectl rollout undo, then I investigate. For example, I deployed an image tag that did not exist. The new pod went into ImagePullBackOff, but because maxUnavailable was zero the old pods kept serving traffic, and a rollback fixed it in seconds."

Follow-up sawaal (chhote answers)

Sawaal	Hinglish mein samjho	English mein bolo
Requests vs limits?	Request = guarantee (scheduler dekhta hai), limit = max	"Requests are reserved and used for scheduling; limits are the maximum. Over the CPU limit the container is throttled; over the memory limit it is OOMKilled."
Exit code 137?	128 + 9 (SIGKILL), yaani memory zyada	"The process was killed with SIGKILL, usually by the OOM killer for exceeding its memory limit."
Service types?	ClusterIP andar, NodePort node pe, LoadBalancer cloud	"ClusterIP is internal only, NodePort also opens a port on every node, LoadBalancer adds a cloud load balancer."
ConfigMap kyun?	Image badle bina settings badlo	"To change configuration without rebuilding the image, so the same image moves from dev to production."
ConfigMap vs Secret?	Secret sensitive data ke liye, base64 (encryption nahi)	"Secrets are for sensitive data. They are only base64-encoded by default, so access must be restricted with RBAC."
Rolling update?	Naya ReplicaSet upar, purana neeche, readiness ke hisaab se	"A new ReplicaSet scales up while the old one scales down, controlled by maxSurge and maxUnavailable and gated by readiness."
Rollback kaise?	<code>kubectl rollout undo</code>	"kubectl rollout undo returns to the previous ReplicaSet revision."
<code>rate()</code> kyun?	Counter sirf badhta hai	"Counters only increase, so rate gives the per-second change, which is what we want to graph."
Helm kyun nahi?	Pehle basics samajhne the	"I wanted to understand every object first. Helm would be my next step to package and version these manifests."
Non-root kyun?	Hack hua to kam power	"If the app is compromised, the attacker has fewer privileges inside the container."
Ek gunicorn worker kyun?	Pods se scale, metrics ek process mein	"In Kubernetes we scale with pods, not processes. It also keeps the Prometheus counters in one process."
Container vs VM?	Container host kernel share karta hai	"A VM virtualises hardware and runs its own kernel; a container is an isolated process sharing the host kernel, using namespaces and cgroups."
Docker layer caching?	Pehle requirements, phir code	"Each instruction is a cached layer. I copy requirements first so dependencies are only reinstalled when they change."
Kubernetes control plane?	API server, etcd, scheduler, controller manager	"The API server is the entry point, etcd stores state, the scheduler places pods, and controllers run reconciliation loops."
Pod Pending kyun hota hai?	Kisi node pe requests jitni jagah nahi	"Usually no node has enough free CPU or memory for its requests. kubectl describe shows the reason."
Aage kya add karoge?	CI/CD, HPA, alerts, Ingress, Helm	"A GitHub Actions pipeline to test and build the image, a HorizontalPodAutoscaler, Alertmanager alerts, an Ingress, and Helm packaging."

"Kya problem aayi thi?" (yeh zaroor poochenge)

Inme se 2-3 apne shabdon mein bolne layak taiyaar rakho:

1. **CRLF line endings:** "Git warned that LF would be replaced by CRLF on Windows. Shell scripts with CRLF fail in Linux containers, so I added a .gitattributes file to force LF endings."
2. **Load test CPU limit tak nahi pahuncha:** "With one sequential client, only one pod was busy at a time, so I never reached the CPU limit. I added parallel clients to the load script."
3. **OOM test reject hua:** "When I lowered only the memory limit, Kubernetes rejected it, because a request cannot be larger than its limit. I had to lower both."
4. **Namespace order:** "kubectl applies files in a folder alphabetically, so I numbered the monitoring files to create the namespace first."
5. **Deploy karte waqt jo asli error aaye,** woh bhi likh lo. Asli experience sabse achha answer hota hai.

Resume description (seedha copy karo)

Kubernetes-Based Application Deployment and Monitoring | Docker, Kubernetes (Minikube), Prometheus, Grafana, Python Flask, Linux, Git

- Containerised a Python Flask web application with a slim, non-root Docker image using layer caching, and deployed it to Kubernetes (Minikube) with Deployment, Service and ConfigMap manifests.
- Configured 3 replicas with rolling updates (zero unavailable pods), readiness and liveness probes, and CPU/memory requests and limits.
- Set up Prometheus with Kubernetes service discovery and RBAC to scrape container CPU/memory (cAdvisor) and custom application metrics, and visualised them on a provisioned Grafana dashboard.
- Validated self-healing and failure handling by deleting pods, failing health checks, deploying a broken image and triggering OOMKills; diagnosed each with kubectl describe, logs and events and recovered with rollbacks.
- Documented architecture, deployment and troubleshooting steps, and added automated tests that validate the application and the consistency of the Kubernetes manifests.

Dhyan rakho: resume pe sirf woh likho jo aapne khud kiya aur samjha hai. Point 4 tabhi likhna jab Part 8 ke experiments khud kar liye hon.

Interview tips

- **"Mujhe nahi pata"** bolna theek hai, phir yeh add karo: **"but I would check it with kubectl describe"** ya **"I would look it up in the documentation"**. Andaza lagake galat bolna bura hai.
- Jawab chhota rakho, aur **example apne project se** do.
- Whiteboard pe architecture diagram bana ke samjhao (Part 1 wala).
- Interviewer GitHub kholega, isliye README aur screenshots achhe hone chahiye.

Part 10 - Aage Kya Seekhein, Glossary aur Cheat Sheet

Is project ko aage kaise badhayein (ek-ek karke)

Har step pichle pe bana hai. Ek baar mein ek hi karo, aur har ek ko GitHub pe alag commit mein daalo.

#	Kya add karein	Kya seekhoge	Mushkil
1	GitHub Actions CI : har push pe tests + <code>docker build</code>	CI/CD, YAML pipelines	Aasaan
2	Image ko GitHub Container Registry pe push	Registries, image tags, auth	Aasaan
3	HorizontalPodAutoscaler (HPA) : CPU zyada to pods apne aap badhein	Autoscaling, metrics-server	Aasaan
4	Ingress : NodePort ki jagah ek proper URL (<code>flask.local</code>)	Ingress controller, routing	Medium
5	Alertmanager : "pod down" ya "CPU 80%+" pe alert	Alerting rules, notifications	Medium
6	kube-state-metrics : pod restarts, replicas ka data	Kubernetes object metrics	Medium
7	Kustomize ya Helm : dev/prod ke liye alag config	Packaging, templating	Medium
8	PersistentVolume for Prometheus/Grafana	Storage, PVC	Medium
9	Loki : logs bhi Grafana mein	Log aggregation	Medium
10	Cloud pe (AWS EKS / Azure AKS free tier) + Terraform	Cloud, Infrastructure as Code	Mushkil

DevOps roadmap (internship ke liye)

1. **Linux**: permissions, processes, systemd, networking commands (`ss`, `ip`, `dig`), shell scripting
2. **Networking**: TCP/IP, DNS, HTTP/HTTPS, load balancers, firewalls
3. **Git**: branching, merge vs rebase, pull requests, conflicts
4. **Docker**: multi-stage builds, Docker Compose, image security scanning
5. **CI/CD**: GitHub Actions ya Jenkins
6. **Kubernetes**: yeh project, phir StatefulSet, DaemonSet, Jobs, Ingress, HPA, RBAC
7. **Monitoring**: Prometheus, Grafana, alerting, logs (Loki/ELK)
8. **Cloud**: ek cloud achhe se (AWS sabse common): EC2, VPC, IAM, S3, EKS
9. **IaC**: Terraform, Ansible
10. **Certification (optional)**: CKA / CKAD (Kubernetes), AWS Cloud Practitioner

Glossary (A to Z)

Term	Matlab
Addon (Minikube)	Ek command se on hone wala extra feature (metrics-server)
Annotation	Object pe extra jaankari, tools ke liye (selection nahi hota)
API server	Kubernetes ka entry gate. kubectl isi se baat karta hai
Build context	<code>docker build</code> ko bheja gaya folder
cAdvisor	kubelet ke andar, containers ka CPU/memory naapta hai
Cardinality	Ek metric ki kitni unique time series hain
cgroups	Linux feature jo process ke resources limit karta hai
ClusterIP	Default Service type, sirf cluster ke andar
ClusterRole	Poore cluster ke liye permissions ki list
ConfigMap	Non-secret config, environment variables ya files ke roop mein
Container	Isolated process (namespaces + cgroups)
Counter	Sirf badhne wala metric
CrashLoopBackOff	Container baar-baar crash, restarts ke beech badhti der
DaemonSet	Har node pe ek pod (is project mein nahi)
Declarative	"Kya chahiye" likho, "kaise" system khud karega
Deployment	ReplicaSet ka manager: replicas, rolling update, rollback
DNS	Naam ko IP mein badalna
DRY	Don't Repeat Yourself: har jaankari ek jagah
emptyDir	Pod ke saath bana aur mita temporary volume
Endpoints	Service ke peeche abhi ke ready pod IPs
etcd	Kubernetes ka database
Exit code 137	SIGKILL (128+9), aksar OOMKilled
Gauge	Upar-neeche hone wala metric
Grafana	Visualization tool, dashboards
Gunicorn	Python ka production web server
Helm	Kubernetes ka package manager (is project mein nahi)
Histogram	Values ko buckets mein ginne wala metric (percentiles)
HPA	Horizontal Pod Autoscaler: load pe pods badhana/ghatana
Idempotent	Kitni baar bhi chalo, result same
Image	Container ka read-only blueprint
imagePullPolicy	Image kab download karni hai (IfNotPresent/Always/Never)
Ingress	HTTP routing cluster ke andar (is project mein nahi)
kubectl	Kubernetes CLI
kubelet	Har node pe agent jo pods chalata aur probes check karta hai
Label	Key-value tag jisse selection hota hai
Layer	Image ka ek hissa, Dockerfile instruction se bana
Limit	Resource ka maximum
Liveness probe	Fail ho to container restart
Minikube	Laptop pe single-node Kubernetes
Namespace (Kubernetes)	Cluster ke andar logical kamra

Term	Matlab
Namespace (Linux)	Process isolation ka kernel feature
Node	Cluster ki ek machine
NodePort	Service jo har node pe port 30000-32767 kholti hai
OOMKilled	Memory limit cross, kernel ne maara
p95	95% requests isse tez thi
Pod	Kubernetes ki sabse chhoti unit, ek ya zyada containers
Port-forward	Laptop ka port seedha ek pod se jodna
Probe	kubelet ka health check
PromQL	Prometheus query language
Prometheus	Metrics collect aur store karne wala tool (pull model)
Provisioning (Grafana)	Data source/dashboard files se apne aap banana
QoS class	Guaranteed / Burstable / BestEffort
rate()	Counter ka per-second badhaav
RBAC	Role-based access control: kaun kya kar sakta hai
Readiness probe	Fail ho to Service se hatao
Reconciliation	Actual state ko desired state jaisa banana, lagatar
Registry	Images store karne ki jagah (Docker Hub)
Relabeling	Prometheus mein discovered targets ko filter/rename karna
ReplicaSet	N pods maintain karta hai
Request	Resource ki guaranteed minimum (scheduling ke liye)
Rolling update	Pods ko dheere-dheere naye version se badalna
Rollback	Pichle version pe wapas (rollout undo)
Scrape	Prometheus ka /metrics se data lena
Secret	Sensitive config (base64, encryption nahi)
Selector	Labels se objects chunna
Service	Pods ke aage stable IP/DNS + load balancing
ServiceAccount	Pod ki identity
Service discovery	Targets ko automatically dhoondhna
SIGTERM / SIGKILL	"Band ho jao" / "Turant maro"
Throttling	CPU limit pe process slow karna
Time series	Naam + labels ki values, time ke saath
venv	Python ka alag environment
WSL2	Windows mein asli Linux kernel
YAML anchor/alias	&naam se define, *naam se reuse

Cheat sheet

Docker

```
docker build -t flask-app:1.0.0 .           # image banao
docker images                               # images list
docker run -d -p 5000:5000 --name t flask-app:1.0.0
docker ps                                   # chal rahe containers
docker logs t                               # logs
docker exec -it t sh                        # andar jao
docker rm -f t                              # hatao
```

Minikube

```
minikube start --driver=docker --cpus=2 --memory=4096
minikube status
minikube image load flask-app:1.0.0
minikube image ls
minikube service <service> [-n namespace] --url
minikube addons enable metrics-server
minikube dashboard
minikube stop
minikube delete
```

kubectl: dekhna

```
kubectl get pods [-A] [-o wide] [-w] [-l app=flask-app]
kubectl get deployment,replicaset,service,endpoints
kubectl get all -n monitoring
kubectl describe pod <naam>
kubectl logs <naam> [--previous] [-f]
kubectl get events --sort-by=.lastTimestamp
kubectl top pods
kubectl get pods --show-labels
```

kubectl: badalna

```
kubectl apply -f k8s/
kubectl delete -f k8s/
kubectl scale deployment flask-app --replicas=5
kubectl set image deployment/flask-app flask-app=flask-app:1.0.1
kubectl rollout status deployment/flask-app
kubectl rollout history deployment/flask-app
kubectl rollout undo deployment/flask-app
kubectl rollout restart deployment/flask-app
kubectl delete pod <naam>
kubectl exec -it <naam> -- sh
kubectl port-forward deployment/flask-app 8080:5000
```

Project scripts

```
./scripts/deploy.sh           # sab kuch deploy
./scripts/load.sh [ms] [n]    # traffic do
./scripts/teardown.sh         # sab hatao
python -m pytest               # tests
python scripts/build_pdf.py   # PDFs dobara banao
```

Git

```
git status
git add .
git commit -m "Message"
git push
git log --oneline
git pull --rebase
```

PromQL

```
up
sum(up{job="app-pods"})
sum by (pod) (rate(flask_http_requests_total[1m]))
sum by (pod) (rate(container_cpu_usage_seconds_total{container="flask-app"}[1m]))
sum by (pod) (container_memory_working_set_bytes{container="flask-app"})
histogram_quantile(0.95, sum by (le) (rate(flask_http_request_duration_seconds_bucket[5m])))
```

Aakhri baat

Yeh project **bada nahi**, lekin **gehra** hai. Interview mein 10 tools ke naam lene se zyada kaam aata hai ek project ko andar se samajhna: kya toota, kyun toota, kaise theek kiya.

Abhi ke 3 kaam:

1. WSL2 + Docker install karo, phir `./scripts/deploy.sh` (Part 7)
2. Part 8 ke 6 experiments **khud** karo, aur jo dikha woh note karo
3. Screenshots GitHub pe daalo

All the best!

Appendix - Poora Source Code

Neeche ki har file seedha repository se li gayi hai, isliye yeh hamesha asli code se match karti hai. Har file ka matlab Part 5 mein samjhaya gaya hai.

app/config.py

```
"""Single source of truth for application settings (DRY).

Every value has a sensible default and can be overridden with an environment
variable, so the SAME image runs unchanged on a laptop, in Docker and in
Kubernetes. Nothing else in the code base hardcodes these values.
"""

import os
from dataclasses import dataclass, field

def _env(name: str, default: str) -> str:
    return os.getenv(name, default)

@dataclass(frozen=True)
class Settings:
    app_name: str = field(default_factory=lambda: _env("APP_NAME", "flask-app"))
    app_version: str = field(default_factory=lambda: _env("APP_VERSION", "dev"))
    app_env: str = field(default_factory=lambda: _env("APP_ENV", "local"))
    port: int = field(default_factory=lambda: int(_env("PORT", "5000")))
    # Inside Kubernetes HOSTNAME is the pod name, which shows which replica answered.
    hostname: str = field(default_factory=lambda: _env("HOSTNAME", "localhost"))

settings = Settings()
```

app/app.py

```
"""A deliberately small Flask app: the focus of this project is DevOps, not features.

Endpoints
GET /           homepage (shows which pod answered)
GET /health     used by Kubernetes readiness and liveness probes
GET /metrics    Prometheus scrape endpoint
GET /work       burns a little CPU so the Grafana graphs move (?ms=100)
POST /fail      makes /health return 503 so you can watch the liveness probe restart the po
d
"""
import time

from flask import Flask, Response, g, jsonify, request
from prometheus_client import CONTENT_TYPE_LATEST, Counter, Histogram, generate_latest

from config import settings

app = Flask(__name__)

REQUESTS = Counter(
    "flask_http_requests_total", "Total HTTP requests", ["method", "endpoint", "status"]
)
LATENCY = Histogram(
    "flask_http_request_duration_seconds", "HTTP request latency in seconds", ["endpoint"]
)

MAX_WORK_MS = 1000

# Per-process flag. Flipped by POST /fail to simulate an app that is "running but broken".
_state = {"healthy": True}

HOMEPAGE = """<!doctype html>
<html><head><title>{name}</title>
<style>body{{font-family:sans-serif;max-width:640px;margin:60px auto;line-height:1.6}}
code{{background:#f3f3f3;padding:2px 6px;border-radius:4px}}</style></head>
<body>
<h1>Hello from {name} &#128075;</h1>
<p>Served by pod <code>{hostname}</code></p>
<p>Version <code>{version}</code> &middot; environment <code>{env}</code></p>
<p>Refresh the page: when the request goes through the Kubernetes Service, a different pod m
ay answer.</p>
<p>Try <a href="/health">/health</a> and <a href="/metrics">/metrics</a>.</p>
</body></html>"""

@app.before_request
def _start_timer():
    g.start = time.perf_counter()

@app.after_request
def _record_metrics(response):
    # Use the route pattern (e.g. "/work"), not the raw URL, so query strings or
    # random paths cannot create unlimited label values (high cardinality).
    endpoint = request.url_rule.rule if request.url_rule else "unmatched"
    if endpoint != "/metrics":
        REQUESTS.labels(request.method, endpoint, response.status_code).inc()
        LATENCY.labels(endpoint).observe(time.perf_counter() - g.start)
    return response
```

```

@app.get("/")
def home():
    return HOMEPAGE.format(
        name=settings.app_name,
        hostname=settings.hostname,
        version=settings.app_version,
        env=settings.app_env,
    )

@app.get("/health")
def health():
    if _state["healthy"]:
        return jsonify(status="ok", pod=settings.hostname)
    return jsonify(status="unhealthy", pod=settings.hostname), 503

@app.get("/metrics")
def metrics():
    return Response(generate_latest(), mimetype=CONTENT_TYPE_LATEST)

@app.get("/work")
def work():
    ms = min(request.args.get("ms", default=100, type=int), MAX_WORK_MS)
    deadline = time.perf_counter() + ms / 1000
    count = 0
    while time.perf_counter() < deadline:
        count += 1
    return jsonify(busy_ms=ms, iterations=count, pod=settings.hostname)

@app.post("/fail")
def fail():
    _state["healthy"] = False
    return jsonify(status="health check will now fail", pod=settings.hostname)

if __name__ == "__main__":
    # Local development only. In the container gunicorn runs the app (see gunicorn.conf.py).
    app.run(host="0.0.0.0", port=settings.port, debug=True)

```

app/gunicorn.conf.py

```

"""Production web server settings. The port comes from config.py (DRY)."""
from config import settings

bind = f"0.0.0.0:{settings.port}"

# One process per container: in Kubernetes we scale by adding pods (replicas),
# not workers. It also keeps the Prometheus counters in a single process.
workers = 1
threads = 4

accesslog = "-" # log requests to stdout so `kubectl logs` shows them

```

app/requirements.txt

```

Flask==3.1.0
prometheus-client==0.21.1
gunicorn==23.0.0

```

Dockerfile

```
# Small official Python base image ("slim" = Debian without extras).
FROM python:3.12-slim

# Version is passed in at build time (scripts/deploy.sh reads it from k8s/deployment.yaml).
ARG APP_VERSION=dev

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    APP_VERSION=${APP_VERSION}

WORKDIR /app

# Copy requirements first: this layer is cached and only rebuilt when dependencies change.
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ .

# Never run as root inside a container.
RUN useradd --uid 10001 --no-create-home appuser
USER 10001

# Documentation only; the real port comes from config.py (PORT, default 5000).
EXPOSE 5000

CMD ["gunicorn", "--config", "gunicorn.conf.py", "app:app"]
```

.dockerignore

```
# Only app/ is needed in the image. Excluding the rest keeps the build context small.
.git
.venv
**/__pycache__
.pytest_cache
tests
docs
k8s
monitoring
scripts
*.md
*.pdf
```

k8s/configmap.yaml

```
# Non-secret configuration, kept outside the image so it can change without a rebuild.
apiVersion: v1
kind: ConfigMap
metadata:
  name: flask-app-config
  labels:
    app: flask-app
data:
  APP_ENV: minikube
```

k8s/deployment.yaml

```
# Deployment = "keep N identical pods of this image running, and replace any that die".
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app
  labels:
    app: flask-app
spec:
  replicas: 3
  # The Deployment owns every pod whose labels match this selector.
  selector:
    matchLabels:
      app: flask-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0 # never drop below 3 ready pods during an update
      maxSurge: 1 # create at most 1 extra pod while updating
  # Template = the blueprint for each pod.
  template:
    metadata:
      labels:
        app: flask-app # must match spec.selector above
      annotations:
        prometheus.io/scrape: "true" # tells our Prometheus config to scrape this pod
    spec:
      securityContext:
        runAsNonRoot: true
      containers:
        - name: flask-app
          # Single source of truth for the image tag: scripts/deploy.sh reads it from here.
          image: flask-app:1.0.0
          imagePullPolicy: IfNotPresent # use the image loaded into Minikube; don't pull from Docker Hub
          ports:
            - name: http # everything below refers to the port by this name, not the number
              containerPort: 5000
          envFrom:
            - configMapRef:
                name: flask-app-config
          resources:
            requests: # what the scheduler reserves for the pod
              cpu: 100m # 0.1 CPU core
              memory: 64Mi
            limits: # hard ceiling: CPU is throttled, memory over the limit = OOMKilled
              cpu: 250m
              memory: 128Mi
          # Readiness: "can this pod receive traffic right now?" Fails -> removed from the Service.
          readinessProbe:
            httpGet: &health-check # YAML anchor: define the check once...
              path: /health
              port: http
            initialDelaySeconds: 3
            periodSeconds: 5
            failureThreshold: 3
          # Liveness: "is this container still alive?" Fails -> kubelet restarts the container.
          livenessProbe:
            httpGet: *health-check # ...and reuse it here (DRY)

            initialDelaySeconds: 10
            periodSeconds: 10
            failureThreshold: 3
```

k8s/service.yaml

```
# Service = one stable name/IP in front of the changing set of pods, with load balancing.
apiVersion: v1
kind: Service
metadata:
  name: flask-app-service
  labels:
    app: flask-app
spec:
  type: NodePort # reachable from outside the cluster on <node-ip>:30080
  selector:
    app: flask-app # send traffic to every READY pod with this label
  ports:
    - name: http
      port: 80 # port inside the cluster: http://flask-app-service
      targetPort: http # the container's named port (5000)
      nodePort: 30080 # port on the Minikube node (allowed range 30000-32767)
```

monitoring/00-namespace.yaml

```
# Keep monitoring tools separate from the application.
# Files in this folder are numbered so `kubectl apply -f monitoring/` creates the namespace
# first.
apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
```

monitoring/10-prometheus.yaml

```
# Prometheus, installed with plain YAML (no Helm) so every line can be read and explained.
#
# 1. ServiceAccount + ClusterRole + ClusterRoleBinding: permission to discover pods/nodes
# 2. ConfigMap: prometheus.yml (WHAT to scrape)
# 3. Deployment: runs the Prometheus server
# 4. Service: exposes the web UI on NodePort 30090
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: prometheus
  namespace: monitoring
---
# Read-only access. "nodes/proxy" lets Prometheus read cAdvisor (container CPU/memory) via the API server.
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: prometheus
rules:
  - apiGroups: [""]
    resources: [nodes, nodes/proxy, nodes/metrics, services, endpoints, pods]
    verbs: [get, list, watch]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: prometheus
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: prometheus
subjects:
  - kind: ServiceAccount
    name: prometheus
    namespace: monitoring
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-config
  namespace: monitoring
data:
  prometheus.yml: |
    global:
      scrape_interval: 15s # how often Prometheus PULLS metrics from every target

    scrape_configs:
      # Prometheus monitors itself.
      - job_name: prometheus
        static_configs:
          - targets: ["localhost:9090"]

      # Container CPU and memory from cAdvisor (built into the kubelet on every node).
      # We go through the API server proxy so it works the same on any cluster.
      - job_name: kubernetes-cadvisor
        scheme: https
        tls_config:
          ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
        authorization:
```

```

    credentials_file: /var/run/secrets/kubernetes.io/serviceaccount/token
  kubernetes_sd_configs:
    - role: node
  relabel_configs:
    - target_label: __address__
      replacement: kubernetes.default.svc:443
    - source_labels: [__meta_kubernetes_node_name]
      target_label: __metrics_path__
      replacement: /api/v1/nodes/${1}/proxy/metrics/cadvisor

  # Application metrics from /metrics on every pod annotated prometheus.io/scrape: "true"
  ".
  # Pods are discovered automatically, so new replicas are scraped without any config change.
  - job_name: app-pods
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
        action: keep
        regex: "true"
      - source_labels: [__meta_kubernetes_pod_container_port_name]
        action: keep
        regex: http # scrape the port named "http" (no port number duplicated here)
      - source_labels: [__meta_kubernetes_namespace]
        target_label: namespace
      - source_labels: [__meta_kubernetes_pod_name]
        target_label: pod
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: prometheus
  namespace: monitoring
  labels:
    app: prometheus
spec:
  replicas: 1
  selector:
    matchLabels:
      app: prometheus
  template:
    metadata:
      labels:
        app: prometheus
    spec:
      serviceAccountName: prometheus
      containers:
        - name: prometheus
          image: prom/prometheus:v2.53.0
          args:
            - --config.file=/etc/prometheus/prometheus.yml
            - --storage.tsdb.path=/prometheus
            - --storage.tsdb.retention.time=2d
          ports:
            - name: web
              containerPort: 9090
          readinessProbe:
            httpGet:
              path: /-/ready

```



```

    port: web
  livenessProbe:
    httpGet:
      path: /-/healthy
    port: web
    initialDelaySeconds: 15
  resources:
    requests:
      cpu: 100m
      memory: 256Mi
    limits:
      cpu: 500m
      memory: 512Mi
  volumeMounts:
    - name: config
      mountPath: /etc/prometheus
    - name: data
      mountPath: /prometheus
  volumes:
    - name: config
      configMap:
        name: prometheus-config
    - name: data
      emptyDir: {} # data is lost when the pod restarts - fine for a learning project
---
apiVersion: v1
kind: Service
metadata:
  name: prometheus
  namespace: monitoring
spec:
  type: NodePort
  selector:
    app: prometheus
  ports:
    - name: web
      port: 9090
      targetPort: web
      nodePort: 30090

```

monitoring/20-grafana.yaml

```
# Grafana, installed with plain YAML (no Helm).
#
# "Provisioning" = Grafana reads config files at startup, so the Prometheus data
# source and our dashboard exist automatically. No clicking needed after a reinstall.
#
# The dashboard itself lives in monitoring/dashboards/flask-app.json (single source).
# scripts/deploy.sh turns it into the "grafana-dashboards" ConfigMap.
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: grafana-provisioning
  namespace: monitoring
data:
  datasources.yaml: |
    apiVersion: 1
    datasources:
      - name: Prometheus
        uid: prometheus          # the dashboard JSON refers to this uid
        type: prometheus
        access: proxy
        url: http://prometheus:9090 # Service name -> cluster DNS
        isDefault: true
  dashboards.yaml: |
    apiVersion: 1
    providers:
      - name: default
        folder: ""
        type: file
        options:
          path: /var/lib/grafana/dashboards
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: grafana
  namespace: monitoring
  labels:
    app: grafana
spec:
  replicas: 1
  selector:
    matchLabels:
      app: grafana
  template:
    metadata:
      labels:
        app: grafana
    spec:
      containers:
        - name: grafana
          image: grafana/grafana:11.1.0
          ports:
            - name: web
              containerPort: 3000
          readinessProbe:
            httpGet:
              path: /api/health
              port: web
          resources:
```

```

    requests:
      cpu: 100m
      memory: 128Mi
    limits:
      cpu: 500m
      memory: 256Mi
  volumeMounts:
    - name: provisioning
      mountPath: /etc/grafana/provisioning/datasources/datasources.yaml
      subPath: datasources.yaml
    - name: provisioning
      mountPath: /etc/grafana/provisioning/dashboards/dashboards.yaml
      subPath: dashboards.yaml
    - name: dashboards
      mountPath: /var/lib/grafana/dashboards
  volumes:
    - name: provisioning
      configMap:
        name: grafana-provisioning
    - name: dashboards
      configMap:
        name: grafana-dashboards
---
apiVersion: v1
kind: Service
metadata:
  name: grafana
  namespace: monitoring
spec:
  type: NodePort
  selector:
    app: grafana
  ports:
    - name: web
      port: 3000
      targetPort: web
      nodePort: 30030

```

monitoring/dashboards/flask-app.json

```
{
  "uid": "flask-app",
  "title": "Flask App on Kubernetes",
  "description": "CPU, memory, traffic and health of the flask-app Deployment",
  "tags": ["flask", "kubernetes"],
  "timezone": "browser",
  "schemaVersion": 39,
  "version": 1,
  "refresh": "10s",
  "time": { "from": "now-30m", "to": "now" },
  "panels": [
    {
      "id": 1,
      "type": "stat",
      "title": "Healthy pods being scraped",
      "description": "Number of flask-app pods whose /metrics endpoint Prometheus can reach (up == 1).",
      "gridPos": { "x": 0, "y": 0, "w": 6, "h": 5 },
      "datasource": { "type": "prometheus", "uid": "prometheus" },
      "targets": [{ "refId": "A", "expr": "sum(up{job=\"app-pods\"})" }],
      "fieldConfig": {
        "defaults": {
          "thresholds": {
            "mode": "absolute",
            "steps": [
              { "color": "red", "value": null },
              { "color": "orange", "value": 2 },
              { "color": "green", "value": 3 }
            ]
          }
        }
      },
      "overrides": []
    },
    {
      "options": { "colorMode": "background", "graphMode": "none", "reduceOptions": { "calcs": ["lastNotNull"] } }
    },
    {
      "id": 2,
      "type": "stat",
      "title": "Total requests / second",
      "gridPos": { "x": 6, "y": 0, "w": 6, "h": 5 },
      "datasource": { "type": "prometheus", "uid": "prometheus" },
      "targets": [{ "refId": "A", "expr": "sum(rate(flask_http_requests_total[$__rate_interval]))" }],
      "fieldConfig": { "defaults": { "unit": "reqps", "decimals": 2 }, "overrides": [] },
      "options": { "graphMode": "area", "reduceOptions": { "calcs": ["lastNotNull"] } }
    },
    {
      "id": 3,
      "type": "stat",
      "title": "Total CPU used by flask-app",
      "gridPos": { "x": 12, "y": 0, "w": 6, "h": 5 },
      "datasource": { "type": "prometheus", "uid": "prometheus" },
      "targets": [{ "refId": "A", "expr": "sum(rate(container_cpu_usage_seconds_total{namespace=\"default\", container=\"flask-app\"}[$__rate_interval]))" }],
      "fieldConfig": { "defaults": { "unit": "none", "decimals": 3, "displayName": "CPU core s" }, "overrides": [] },
      "options": { "graphMode": "area", "reduceOptions": { "calcs": ["lastNotNull"] } }
    }
  ]
}
```

```

    "id": 4,
    "type": "stat",
    "title": "Total memory used by flask-app",
    "gridPos": { "x": 18, "y": 0, "w": 6, "h": 5 },
    "datasource": { "type": "prometheus", "uid": "prometheus" },
    "targets": [ { "refId": "A", "expr": "sum(container_memory_working_set_bytes{namespace=
\\default\\, container=\\flask-app\\})" } ],
    "fieldConfig": { "defaults": { "unit": "bytes" }, "overrides": [ ] },
    "options": { "graphMode": "area", "reduceOptions": { "calcs": [ "lastNotNull" ] } }
  },
  {
    "id": 5,
    "type": "timeseries",
    "title": "CPU usage per pod (cores)",
    "description": "Compare with the CPU limit in k8s/deployment.yaml. A flat line at the
limit means the pod is being throttled.",
    "gridPos": { "x": 0, "y": 5, "w": 12, "h": 9 },
    "datasource": { "type": "prometheus", "uid": "prometheus" },
    "targets": [
      {
        "refId": "A",
        "expr": "sum by (pod) (rate(container_cpu_usage_seconds_total{namespace=\\default\\
, container=\\flask-app\\}[\\$__rate_interval]))",
        "legendFormat": "{{pod}}"
      }
    ],
    "fieldConfig": { "defaults": { "unit": "none", "decimals": 3, "min": 0 }, "overrides":
[ ] }
  },
  {
    "id": 6,
    "type": "timeseries",
    "title": "Memory usage per pod (working set)",
    "description": "Compare with the memory limit in k8s/deployment.yaml. Going above it g
ets the container OOMKilled.",
    "gridPos": { "x": 12, "y": 5, "w": 12, "h": 9 },
    "datasource": { "type": "prometheus", "uid": "prometheus" },
    "targets": [
      {
        "refId": "A",
        "expr": "sum by (pod) (container_memory_working_set_bytes{namespace=\\default\\, c
ontainer=\\flask-app\\})",
        "legendFormat": "{{pod}}"
      }
    ],
    "fieldConfig": { "defaults": { "unit": "bytes", "min": 0 }, "overrides": [ ] }
  },
  {
    "id": 7,
    "type": "timeseries",
    "title": "Requests per second by pod",
    "description": "Shows the Service spreading traffic across replicas.",
    "gridPos": { "x": 0, "y": 14, "w": 12, "h": 9 },
    "datasource": { "type": "prometheus", "uid": "prometheus" },
    "targets": [
      {
        "refId": "A",
        "expr": "sum by (pod) (rate(flask_http_requests_total[\\$__rate_interval]))",
        "legendFormat": "{{pod}}"
      }
    ]
  }

```

```

    ],
    "fieldConfig": { "defaults": { "unit": "reqps", "min": 0 }, "overrides": [] }
  },
  {
    "id": 8,
    "type": "timeseries",
    "title": "95th percentile response time",
    "gridPos": { "x": 12, "y": 14, "w": 12, "h": 9 },
    "datasource": { "type": "prometheus", "uid": "prometheus" },
    "targets": [
      {
        "refId": "A",
        "expr": "histogram_quantile(0.95, sum by (le, endpoint) (rate(flask_http_request_uration_seconds_bucket[ $__rate_interval ])))",
        "legendFormat": "{{endpoint}}"
      }
    ],
    "fieldConfig": { "defaults": { "unit": "s", "min": 0 }, "overrides": [] }
  }
]
}

```

scripts/deploy.sh

```

#!/usr/bin/env bash
# Build the image and deploy the app + monitoring to Minikube in one go.
# Run from Git Bash (Windows), WSL or any Linux/macOS terminal: ./scripts/deploy.sh
#
# DRY: the image name and tag are read from k8s/deployment.yaml, so the version
# is written in exactly one place.
set -euo pipefail
cd "$(dirname "$0")/.."

IMAGE=$(awk ' $1 == "image:" {print $2; exit}' k8s/deployment.yaml)
VERSION=${IMAGE##*:}

step() { printf '\n==> %s\n' "$*"; }

step "Starting Minikube (skipped if it is already running)"
minikube status >/dev/null 2>&1 || minikube start --driver=docker --cpus=2 --memory=4096
minikube addons enable metrics-server >/dev/null # enables `kubectl top`

step "Building $IMAGE"
docker build --build-arg APP_VERSION="$VERSION" -t "$IMAGE" .

step "Loading $IMAGE into Minikube"
minikube image load "$IMAGE"

step "Deploying the application"
kubectl apply -f k8s/

step "Deploying Prometheus and Grafana"
kubectl apply -f monitoring/
# Generate the dashboard ConfigMap from the JSON file (create-or-update).
kubectl create configmap grafana-dashboards -n monitoring \
  --from-file=monitoring/dashboards/ --dry-run=client -o yaml | kubectl apply -f -

step "Waiting for everything to become ready"
kubectl rollout status deployment/flask-app --timeout=120s
kubectl rollout status deployment/prometheus -n monitoring --timeout=180s
kubectl rollout status deployment/grafana -n monitoring --timeout=180s

step "Done. Open the services (each command keeps a tunnel open; use a separate terminal):"
cat <<'EOF'
  minikube service flask-app-service
  minikube service prometheus -n monitoring
  minikube service grafana -n monitoring      # login admin / admin
EOF

```

scripts/load.sh

```
#!/usr/bin/env bash
# Generate traffic from INSIDE the cluster so the Grafana graphs move.
# Requests go through the Service, so you can watch the load spread across pods.
#
# Usage: ./scripts/load.sh [busy-ms-per-request] [parallel-clients]
#   ./scripts/load.sh           light load (50 ms, 1 client)
#   ./scripts/load.sh 500 6     heavy load: pods hit their CPU limit
# Stop with Ctrl+C (the pod is deleted automatically).
set -euo pipefail

MS=${1:-50}
CLIENTS=${2:-1}
# winpty is needed for interactive kubectl in Git Bash on Windows.
TTY=$(command -v winpty >/dev/null 2>&1 && echo winpty || true)

$TTY kubectl run load-generator --rm -it --restart=Never --image=busybox:1.36 -- /bin/sh -c
"
  for i in $(seq $CLIENTS); do
    while true; do wget -q -O /dev/null 'http://flask-app-service/work?ms=$MS'; done &
  done
  echo 'Sending load with $CLIENTS client(s), $MS ms per request. Ctrl+C to stop.'
  wait"
```

scripts/teardown.sh

```
#!/usr/bin/env bash
# Remove everything deploy.sh created (the Minikube cluster itself is kept).
set -euo pipefail
cd "$(dirname "$0")/.."

kubectl delete -f k8s/ --ignore-not-found
kubectl delete -f monitoring/ --ignore-not-found # deleting the namespace also removes grafana-dashboards
echo "Removed. Run 'minikube stop' to pause the cluster or 'minikube delete' to remove it."
```

tests/test_app.py

```
import pytest

import app as app_module

@pytest.fixture
def client():
    app_module._state["healthy"] = True
    app_module.app.config["TESTING"] = True
    with app_module.app.test_client() as client:
        yield client

def test_homepage_shows_app_name(client):
    response = client.get("/")
    assert response.status_code == 200
    assert b"flask-app" in response.data

def test_health_is_ok(client):
    response = client.get("/health")
    assert response.status_code == 200
    assert response.get_json()["status"] == "ok"

def test_fail_makes_health_return_503(client):
    client.post("/fail")
    assert client.get("/health").status_code == 503

def test_metrics_counts_requests(client):
    client.get("/health")
    body = client.get("/metrics").get_data(as_text=True)
    assert 'flask_http_requests_total{endpoint="/health",method="GET",status="200"}' in body

def test_work_is_capped(client):
    response = client.get("/work?ms=999999")
    assert response.get_json()["busy_ms"] == app_module.MAX_WORK_MS
```


tests/test_manifests.py

```
"""Offline checks that the Kubernetes and monitoring files agree with each other.

These catch the classic "selector does not match labels" mistakes without needing a cluster.
"""
import json
from pathlib import Path

import pytest

yaml = pytest.importorskip("yaml")

ROOT = Path(__file__).resolve().parent.parent

def load_docs(folder):
    docs = []
    for path in sorted((ROOT / folder).glob("*.yaml")):
        docs += [d for d in yaml.safe_load_all(path.read_text()) if d]
    return docs

def find(docs, kind, name):
    return next(d for d in docs if d["kind"] == kind and d["metadata"]["name"] == name)

APP = load_docs("k8s")
MON = load_docs("monitoring")

def test_deployment_selector_matches_pod_labels():
    for dep in [d for d in APP + MON if d["kind"] == "Deployment"]:
        selector = dep["spec"]["selector"]["matchLabels"]
        labels = dep["spec"]["template"]["metadata"]["labels"]
        assert selector.items() <= labels.items(), dep["metadata"]["name"]

def test_every_service_selects_a_deployment_and_a_named_port():
    deployments = [d for d in APP + MON if d["kind"] == "Deployment"]
    for svc in [d for d in APP + MON if d["kind"] == "Service"]:
        selector = svc["spec"]["selector"]
        matches = [
            d for d in deployments
            if selector.items() <= d["spec"]["template"]["metadata"]["labels"].items()
        ]
        assert matches, f"Service {svc['metadata']['name']} selects no pods"
        port_names = {p["name"] for c in matches[0]["spec"]["template"]["spec"]["containers"]}
        for p in c["ports"]:
            for port in svc["spec"]["ports"]:
                assert port["targetPort"] in port_names

def test_probes_share_one_health_check():
    container = find(APP, "Deployment", "flask-app")["spec"]["template"]["spec"]["containers"]
    assert container["readinessProbe"]["httpGet"] == container["livenessProbe"]["httpGet"]
    assert container["resources"]["requests"] and container["resources"]["limits"]

def test_prometheus_config_is_valid_yaml_with_app_job():
    config = yaml.safe_load(find(MON, "ConfigMap", "prometheus-config")["data"]["prometheus."])
```

```

yaml""))
assert "app-pods" in {job["job_name"] for job in config["scrape_configs"]}

def test_dashboard_uses_provisioned_datasource():
    provisioning = find(MON, "ConfigMap", "grafana-provisioning")["data"]
    uid = yaml.safe_load(provisioning["datasources.yaml"])["datasources"][0]["uid"]
    dashboard = json.loads((ROOT / "monitoring/dashboards/flask-app.json").read_text())
    assert {p["datasource"]["uid"] for p in dashboard["panels"]} == {uid}

```